DEVELOPER DOCUMENTATION

HORIZON GRID

Source Code / Developer Documentation

Version: 0.2.5
Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

Project Overview, Technology Stack, and Repository Structure

1. What This Project Is

HORIZON GRID is a self-hosted SOC (Security Operations Center) workbench for investigating **IOCs** (Indicators of Compromise — an IP address, domain, URL, file hash, CVE, or similar artifact). The backend's own FastAPI app description (`backend/app/main.py:20-22` , shown in the Swagger UI at `/docs`) describes the core loop as "single-search IOC lookup across dozens of providers, correlated and summarized by a local Ollama model (or AWS Bedrock/Gemini/Anthropic/Groq, configurable via AI_BACKEND)."

Concretely, one investigation does the following, all reachable from a single API call (`POST /api/v1/lookup/stream`):

1. A raw string is classified into a typed IOC (`backend/app/ioc/detector.py`).
2. The IOC is fanned out concurrently to as many of ~18 registered threat-intelligence providers as support that IOC type (`backend/app/providers/orchestrator.py`).
3. A deterministic correlation engine extracts relationships between the seed IOC and anything the providers returned (`backend/app/correlation/engine.py`).
4. A deterministic, non-AI evidence ledger is built from the raw provider data and correlation edges (`backend/app/evidence/builder.py`) — this ledger is what every later AI explanation must cite back to.
5. An AI backend (one of eleven interchangeable options, hot-swappable without a restart) summarizes each provider's result and then produces one final verdict/risk-score assessment grounded in the evidence (`backend/app/ai/service.py`).
6. Every step streams back to the browser as Server-Sent Events, and the whole run — provider results, AI summaries, correlation edges, evidence, final assessment — is persisted to PostgreSQL.

Beyond the core lookup, the platform layers analyst workflow on top: a persisted evidence ledger with on-demand AI explanations (why-malicious, disagreement, false-positive, red-team challenge, next-actions, intelligence gaps, score explanation, free-form "Copilot" Q&A), threat-hunting query generation (Sigma/Splunk/KQL/etc.), an IOC "basket" scratch space with AI-assisted comparison, case management, JWT-based auth with three roles, and a DB-backed runtime configuration system that lets an admin change AI/provider credentials from the UI with no container restart.

This is a real, running codebase — not a prototype: it ships as a Windows-installable product (Inno Setup + a PowerShell setup wizard over Docker Compose), has an async backend test suite (`backend/app/tests/`), and has two parallel deployment paths (Docker Compose and Kubernetes via `k8s/`).

2. Technology Stack

2.1 Backend

Layer	Choice	Version (pinned)	Source
Language / runtime	Python	3.12 (`python:3.12-slim` base image)	`backend/Dockerfile`
Web framework	FastAPI	0.115.0	`backend/requirements.txt`
ASGI server	Uvicorn (`[standard]`)	0.30.6	`backend/requirements.txt`
Validation / settings	Pydantic, pydantic-settings	2.9.2, 2.5.2	`backend/requirements.txt`
ORM	SQLAlchemy (`[asyncio]`)	2.0.35	`backend/requirements.txt`
Postgres driver	asyncpg	0.29.0	`backend/requirements.txt`
Migrations	Alembic	1.13.2	`backend/requirements.txt`
Cache / broker client	redis-py	5.0.8	`backend/requirements.txt`
HTTP client	httpx	0.27.2	`backend/requirements.txt`
Retry logic	tenacity	9.0.0	`backend/requirements.txt`
Graph DB driver (provisioned, unused — see Database chapter)	neo4j	5.24.0	`backend/requirements.txt`
Background jobs	Celery	5.4.0	`backend/requirements.txt`
Auth / JWT	python-jose (`[cryptography]`)	3.3.0	`backend/requirements.txt`
Credential encryption	cryptography	43.0.1	`backend/requirements.txt`
Password hashing	passlib, bcrypt	1.7.4, 4.0.1	`backend/requirements.txt`
Cloud SDK (Bedrock)	boto3	1.35.24	`backend/requirements.txt`
Search client (provisioned, unused)	opensearch-py	2.7.1	`backend/requirements.txt`
OSINT feed parsing	feedparser, beautifulsoup4, lxml	6.0.11, 4.12.3, 5.3.0	`backend/requirements.txt`
WHOIS	python-whois	0.9.6	`backend/requirements.txt`
Logging	structlog	24.4.0	`backend/requirements.txt`
Metrics	prometheus-fastapi-instrumentator	7.0.0	`backend/requirements.txt`
Testing	pytest, pytest-asyncio, pytest-cov, respx	8.3.3, 0.24.0, 5.0.0, 0.21.1	`backend/requirements.txt`

2.2 Frontend

Layer	Choice	Version (pinned)	Source
Runtime (container)	Node.js	20 (`node:20-alpine` base image)	`frontend/Dockerfile`
Framework	Next.js (App Router)	14.2.15	`frontend/package.json`
UI library	React / ReactDOM	18.3.1	`frontend/package.json`
Language	TypeScript	5.6.2	`frontend/package.json`
Styling	Tailwind CSS	3.4.12	`frontend/package.json`
Client state	Zustand	4.5.5	`frontend/package.json`
Charts	Recharts	2.12.7	`frontend/package.json`
Graph visualization	react-force-graph-2d	1.25.4	`frontend/package.json`
UI primitives	Radix UI (`react-tabs`, `react-dialog`, `react-progress`, `react-slot`, `react-tooltip`)	1.0.x–1.1.x	`frontend/package.json`
Class variants / utility CSS	class-variance-authority, clsx, tailwind-merge	0.7.0, 2.1.1, 2.5.2	`frontend/package.json`
Icons	lucide-react	0.446.0	`frontend/package.json`
Test runner (configured, unused)	vitest	2.1.1	`frontend/package.json` — `"test": "vitest run"` is declared, but no `*.test.*` / `*.spec.*` files exist anywhere under `frontend/`; the tooling is present, no frontend automated tests currently exist.

2.3 Datastores and infrastructure

Component	Image	Actually used?
PostgreSQL	`postgres:16-alpine`	Yes — the sole system of record; every lookup, provider result, AI summary, correlation edge, evidence item, case, and user row lives here.
Redis	`redis:7-alpine`	Yes — three roles on separate logical DB indexes: provider-result cache + rate limiter (`/0`), Celery broker (`/1`), Celery result backend (`/2`).
Neo4j	`neo4j:5-community` + APOC	Container runs and a driver is in `requirements.txt`, but no Cypher query or driver instantiation exists anywhere in the application code — provisioned only.
OpenSearch	`opensearchproject/opensearch:2.17.0`	Container runs and `opensearch-py` is in `requirements.txt`, but no index/search client code exists — provisioned only.
Celery worker + beat	same backend image, `celery_worker` / `celery_beat` services	Yes, for exactly one scheduled job: hourly OSINT re-crawl (`run_osint_crawl`). The interactive `/lookup/stream` path never touches Celery.

Orchestration is defined in `docker-compose.yml` (development, eight containers total including `frontend`) and layered with `docker-compose.prod.yml` for production (strips dev bind-mounts, switches to production start commands). A second, independent deployment path exists in `k8s/` — a Kustomize-based Kubernetes manifest set (`k8s/base/`) with StatefulSets for `postgres` / `neo4j` / `opensearch`, Deployments for `backend` / `frontend` / `celery`, and an Ingress resource.

3. Repository Structure

```
ioc-intel-platform/
├── .env / .env.example          Backend credential/URL defaults (Settings source); real .env is never committed
├── docker-compose.yml           Dev topology: postgres, redis, neo4j, opensearch, backend, celery_worker, celery_beat, frontend
├── docker-compose.prod.yml      Production overlay (no bind-mounts, prod start commands) — used by the Windows installer
├── README.md                    Top-level project summary
├── *_REPORT.md                  Root-level engineering session reports (bug/fix/test write-ups; not shipped documentation)
├── test-provider-pipeline.sh    Ad hoc shell script that re-runs provider connection tests against real container env
│
├── backend/                     FastAPI application (Python 3.12)
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── alembic/                 Schema migrations
│   │   ├── env.py               Async-engine Alembic runner; imports app.models.Base
│   │   └── versions/            4 linear migrations (initial_schema → evidence/basket/case → provider_runtime_config/audit → final_assessment_records)
│   └── app/
│       ├── main.py              App assembly: router registration, CORS, startup hook (runtime-config seeding), /health, /metrics
│       ├── api/routes/          10 route modules — one per resource area (see §4)
│       ├── core/                config.py (Settings), runtime_config.py (DB-backed config), runtime_context.py (ContextVar overrides), db.py, cache.py, crypto.py
│       ├── auth/                security.py (bcrypt/JWT), rbac.py (get_current_user, require_permission)
│       ├── ioc/                 types.py (IOCType enum), detector.py (regex-cascade classifier)
│       ├── providers/           ~18 threat-intel connectors + base.py, orchestrator.py, registry.py, connection_test.py, stubs/
│       ├── ai/                  11 AI-backend clients + service.py, analysis_service.py, hunting_service.py, schemas, connection_test.py
│       ├── correlation/         engine.py — pure fact/relationship extraction, no I/O
│       ├── evidence/            builder.py, loaders.py, pivot.py — the citable, deterministic evidence ledger
│       ├── crawler/             OSINT collector provider + 4 sub-source modules (github, reddit, rss_news, pastebin_search)
│       ├── workers/             celery_app.py, tasks.py — the one hourly OSINT re-crawl job
│       ├── models/              SQLAlchemy ORM: user, lookup, evidence, basket, case, runtime_config, base
│       ├── schemas/              Pydantic request/response DTOs (lookup, basket, case, auth)
│       └── tests/               unit/ (18 files) + integration/ (3 files), pytest + pytest-asyncio + respx
│
├── frontend/                    Next.js 14 application (TypeScript)
│   ├── Dockerfile
│   ├── package.json
│   ├── app/                     App Router pages: home, lookup/new, lookup/[id], basket, cases, cases/[id], providers, login, register
│   ├── components/
│   │   ├── dashboard/           ~22 investigation-workspace widgets (ProviderCardGrid, RelationshipGraph, EvidencePanel, HuntingCenterPanel, ...)
│   │   └── ui/                  shadcn-style primitives (button, card)
│   └── lib/                     api.ts (all backend calls + SSE consumer), types.ts (manual mirror of backend Pydantic schemas), utils.ts, aiPrompt.ts
│
├── docs/                        Hand-written engineering reference docs (ARCHITECTURE.md, API.md, DATA_MODEL.md, PROVIDERS.md, DEPLOYMENT.md, TESTING.md, TROUBLESHOOTING.md
├── documentation/               Generated product-documentation pipeline (this chapter's home)
│   ├── DOCUMENTATION_SOURCE/    Markdown source chapters (tech-*.md, user-*.md, dev-*.md)
│   ├── ARCHITECTURE_DIAGRAMS/   Rendered diagram PNGs referenced by [FIGURE: ...] placeholders
│   ├── SCREENSHOTS/             Product screenshots for the user manual
│   └── build/                   Node.js scripts assembling source Markdown into PDF/DOCX deliverables
│
├── k8s/                         Kustomize-based Kubernetes manifests (alternative to Docker Compose)
├── windows/                     Windows packaging: Inno Setup script + PowerShell setup wizard + service/backup/diagnostic scripts
└── release/                     Built installer artifact (.exe) + checksums
```

4. The Backend API Surface, at a Glance

`backend/app/main.py` registers ten route modules under the global prefix `/api/v1` (`settings.api_v1_prefix`), in this order:

Module	Base path	Endpoint count	Covers
`auth.py`	`/auth`	4	register / login / refresh / me
`lookup.py`	`/lookup`	5	the core SSE investigation stream, reanalyze, assessments, get/list
`providers.py`	`/providers`	2	provider health, live credential test
`ai_config.py`	`/ai`	2	AI backend live test, model-list discovery
`analysis.py`	`/lookup/{id}/analysis`	10	evidence + 9 AI explanation endpoints (why, what-is-this, disagreement, false-positive, challenge, next-actions, gaps, score-explanation, copilot)
`hunting.py`	`/lookup/{id}`	2	hunting-query package, detection-rule draft
`pivot.py`	`/lookup/{id}/pivots`	1	deterministic pivot ranking
`basket.py`	`/basket`	5	per-analyst scratch list + AI comparison
`cases.py`	`/cases`	8	case CRUD, IOCs, notes
`runtime.py`	`/runtime`	10	DB-backed AI/provider config, activation, audit log

Total: 49 endpoints, all gated by `require_permission(...)` except the three public auth endpoints and `/auth/me` (identity-only). The full request/response contract for every one of these is documented in the API Reference chapter of this package; this chapter is concerned only with orienting a new engineer to where that code lives.

5. Files a New Engineer Should Read First

The following ~26 files, read roughly in this order, cover the entire system end to end. This list reflects the actual dependency order of the codebase, not an arbitrary tour.

Entry point and configuration

1. `backend/app/main.py` — app assembly, the full router list, and the startup hook; the map of everything else.
2. `backend/app/core/config.py` — every environment-driven setting (`Settings`, `@lru_cache`'d `get_settings()`); read before anything else that touches a credential or URL default.
3. `backend/app/core/runtime_config.py` — the DB-backed configuration system that overrides #2 at runtime without a restart; central to understanding how AI backends and provider credentials are actually managed in production.
4. `backend/app/core/runtime_context.py` — the small but easy-to-miss `ContextVar` mechanism that makes per-investigation provider-credential overrides safe under concurrent requests.

The investigation pipeline — the actual product

5. `backend/app/api/routes/lookup.py` — the single most important file: the whole SSE-streamed investigation lifecycle (`detect → fan-out → correlate → summarize → assess → persist`) lives here.
6. `backend/app/providers/base.py` — the `BaseProvider` contract every connector implements; read before any individual provider file.
7. `backend/app/providers/orchestrator.py` — how one IOC fans out to N providers concurrently, with caching, retry, and timeout.
8. `backend/app/providers/registry.py` — the flat list of all ~18 registered provider singletons; the extensibility seam for adding a new source.
9. `backend/app/providers/virustotal.py` — the most complete, documented reference connector to model new providers on.
10. `backend/app/ioc/detector.py` — how a raw string becomes a typed `IOCType`; determines which providers even run for a given input.
11. `backend/app/ioc/types.py` — the `IOCType` enum and `HASH_TYPES` / `NETWORK_TYPES` sets that every other module keys off of.
12. `backend/app/correlation/engine.py` — the pure, I/O-free function that turns N provider results into a relationship graph and provider-agreement data.
13. `backend/app/evidence/builder.py` — converts providers + correlation into the citable evidence ledger that grounds every later AI claim; the anti-hallucination foundation of the product.
14. `backend/app/ai/service.py` — the two-stage AI pipeline (per-provider summary → final assessment), the 3-tier backend-resolution logic (`_get_ai_client`), and the correlation-grounding cross-check.
15. `backend/app/ai/schemas.py` — the structured-output contracts every AI backend must satisfy; explains why the schemas use real `Literal`/enum types rather than prose.
16. `backend/app/ai/analysis_service.py` — the "explain this completed lookup" AI functions and the evidence-id grounding/stripping mechanism.

Data model and authorization

17. `backend/app/models/lookup.py` — the core persisted entities: `IOCLookup`, `ProviderResultRecord`, `AISummaryRecord`, `CorrelationEdgeRecord`, `FinalAssessmentRecord`.
18. `backend/app/models/evidence.py` — the evidence-ledger schema (`EvidenceItem`).
19. `backend/app/models/runtime_config.py` — the runtime provider/AI configuration schema backing #3.
20. `backend/app/models/user.py` — `Role` and `ROLE_PERMISSIONS`; the entire authorization matrix in one file.
21. `backend/app/auth/rbac.py` — how `get_current_user` / `require_permission(...)` actually gate every route.

Frontend

22. `frontend/lib/api.ts` — every backend interaction, including the hand-rolled SSE consumer (`streamLookup`) and 401-refresh-retry logic; the frontend's single point of coupling to the backend contract.
23. `frontend/lib/types.ts` — a manually-maintained TypeScript mirror of the backend's Pydantic schemas; read alongside #15/#17 to see the full contract from both sides.
24. `frontend/app/lookup/new/page.tsx` — the live-investigation composition root; shows how every dashboard widget consumes the SSE event stream.
25. `frontend/app/lookup/[id]/page.tsx` — the completed-lookup equivalent, useful as a diff against #24 for what the API reshapes once a lookup is persisted.

Operations and packaging

26. `docker-compose.yml` — the full local runtime topology and the `host.docker.internal` Ollama networking detail.
27. `windows/wizard/Setup-Wizard.ps1` (with `windows/scripts/Common.ps1`) — how the stack becomes an installable Windows product; explains the Program Files/ProgramData split and the live-provider-testing UX that mirrors `providers/connection_test.py`.

6. Cross-Cutting Facts Worth Internalizing Before Reading Further

- Every provider is a **module-level singleton**; per-investigation credential and enabled/disabled overrides flow through a `ContextVar` (`core/runtime_context.py`), never through mutated instance state — this is what makes concurrent investigations with different provider configs safe.
- Every AI call site resolves its backend the same three-tier way (`_get_ai_client`): explicit override → active DB-backed runtime config → legacy `Settings` fallback. This pattern repeats identically across `ai/service.py`, `ai/analysis_service.py`, and `ai/hunting_service.py`.
- Nothing the AI produces is trusted at face value: `evidence/builder.py` generates ground truth with zero AI involvement, and every AI explanation function strips citations that don't reference a real evidence id, a real correlation edge, or a real provider id.
- The frontend has no independent business logic. `frontend/lib/api.ts` is a pure pass-through to the FastAPI routes, and `frontend/lib/types.ts` is a hand-maintained mirror of the backend's Pydantic models — the two must be kept in sync manually; there is no code-generation step between them.
- Two containers in every deployment — `neo4j` and `opensearch` — are provisioned and running but have no consuming application code today (see the Database Architecture chapter for the full verification). Treat any reference to a "Neo4j-backed graph" or "OpenSearch-backed search" in older docstrings or comments as aspirational, not implemented.

Figure 1 — Diagram: 6. Cross-Cutting Facts Worth Internalizing Before Reading Further

Diagram: Repository and stack overview — solid lines are active data paths; dotted lines mark containers that run but have no consuming application code today.

Frontend Architecture

The frontend is a single **Next.js 14.2.15** application (React 18.3.1, TypeScript, App Router) under `frontend/`. It has no independent business logic: every piece of data it renders was either fetched from the backend's `/api/v1/*` HTTP surface or derived client-side from data the backend already returned. This section documents the four things a developer needs to understand to work on it: the route tree under `frontend/app`, the widget library under `frontend/components/dashboard`, the single API client module `frontend/lib/api.ts`, and the type mirror `frontend/lib/types.ts` -- plus the SSE (Server-Sent Events) consumer that drives the live investigation view, which is the most architecturally significant piece of client code in the product.

Route inventory (`frontend/app`, App Router)

Every route below is a real file under `frontend/app`. All page components are marked `"use client"` -- there are no server components or server actions in this codebase; every page fetches its data in a `useEffect` after mount, using the token stored in `localStorage`.

Route	File	Purpose
`/`	`app/page.tsx`	Home page: search box submits to `/lookup/new?value=<encoded IOC>`; redirects to `/login?next=...` if not logged in. Renders `AiQuickSwitch` for logged-in users plus four example IOC quick-fill buttons.
`/login`	`app/login/page.tsx`	Email/password sign-in; calls `login()`, then routes to `?next=` (default `/`). Wrapped in `<Suspense>` (reads `useSearchParams()`).
`/register`	`app/register/page.tsx`	Registration form calling `register()`; per the backend's `auth.py`, the first-ever user becomes `admin` and every registration attempt after that is rejected with `403` -- the page itself now says as much, pointing anyone past bootstrap at the Administration page.
`/lookup/new`	`app/lookup/new/page.tsx`	**Composition root for a live investigation.** Reads `?value=`, opens the SSE stream via `streamLookup()`, renders the full two-column dashboard as events arrive. Detailed below.
`/lookup/[id]`	`app/lookup/[id]/page.tsx`	Same dashboard for an *already-completed* lookup: one `getLookup(id)` call instead of SSE, with a `RawLookupDetail` reshaping step to backfill fields the persisted response omits (`ioc_value`, `ioc_type`, `fetched_at`, `from_cache`).
`/basket`	`app/basket/page.tsx`	IOC Basket: list/add/remove/clear the caller's own items, "Investigate All", multi-select "Compare" (`POST /basket/compare`).
`/cases`	`app/cases/page.tsx`	Case list: create (title/description/severity) or open a case; status badges color-coded per `CaseStatus`.
`/cases/[id]`	`app/cases/[id]/page.tsx`	Case detail: attached IOCs, analyst notes, reports.
`/providers`	`app/providers/page.tsx`	"Manage Providers" -- UI for the runtime-config system (`backend/app/core/runtime_config.py`). Four Radix `Tabs`: AI Providers, IOC Providers, Audit Log, Network Access. Every action takes effect immediately, no backend restart.
`/admin`	`app/admin/page.tsx`	**Administration console** -- multi-admin user management. Guarded twice: `isLoggedIn()` (redirect to `/login`) then `getCurrentUser().role !== "admin"` (redirect to `/`, since this is UX only -- the real enforcement is server-side `require_permission("user:manage")` on every `/api/v1/admin/*` route). Four Radix `Tabs`: Overview (account-count stat cards + recent logins from `getUserStats()`), Users (`UsersManagementPanel`, below), Roles & Permissions (read-only matrix from `getRoles()`), Audit Log (same `getAuditLog()`/rendering block as `/providers`' Audit Log tab -- one shared table, not a duplicate).

`app/layout.tsx` is the root layout: it forces `<html class="dark">` (the product has no light theme), sets the page `<title>`/`<meta description>`, and imports `app/globals.css` (the CSS-variable-based dark theme that every color helper in `lib/utils.ts` and the relationship graph read from). There is no `app/api/` directory -- Next.js is used purely as a client-rendering shell; it has zero server-side API routes of its own, and every backend call resolves its base URL through `getApiUrl()` (`lib/api.ts:46-51`, detailed below), not a single fixed address.

`/lookup/new` wraps its inner component in a `LookupNewPageKeyed` wrapper that sets `key={rawValue}` (`app/lookup/new/page.tsx`), so pivoting to a new IOC via `TopSearchBar` -- which pushes a new `?value=` onto the same route -- fully unmounts and remounts the investigation view rather than reusing React state, guaranteeing a fresh SSE stream and no state bleed (provider results, event log, evidence highlights) from the previous IOC.

Component organization (`frontend/components/dashboard`)

Twenty-five `.tsx` files, all in one flat directory (no further subfolders). Most are consumed by the two lookup composition roots above; `UsersManagementPanel` (added for the Administration console) is the one exception, consumed only by `/admin`. They fall into five functional groups:

Live-stream / core investigation widgets (populate incrementally as SSE events arrive): `ThreatScoreGauge` (SVG risk-score arc, reads `finalAssessment.risk`); `ProviderCardGrid`/`ProviderCard` (one card per provider result plus its AI summary, sorted `status === "ok"` first); `ProviderProgressTracker` (sidebar counter of providers responded vs. `totalExpected`, computed from `getProviderHealth()` filtered by the detected type's `supported_types`); `FinalAssessmentPanel` (renders the consolidated `FinalAssessment`); `RelationshipGraph` (correlation-graph visualization, detailed below); `MitreMatrix`, `DetectionRulesPanel`, `RecommendedActionsPanel` (render `finalAssessment.mitre_mappings`, `.detection_rules`, and the recommendation arrays).

Post-completion analysis widgets -- gated behind `isDone && lookupId` (or `isCompleted`), since their endpoints require `status == "completed"`: `VerdictAnalysisPanel` (on-demand why/what-is/score/disagreement/false-positive/challenge calls, each with a "Show Receipts" affordance); `EvidencePanel` (raw `EvidenceItem[]` ledger, accepts a `highlightIds` prop that `ShowReceiptsLink` populates); `PivotPanel` (`GET /lookup/{id}/pivots`); `HuntingCenterPanel` (hunting-query/detection-rule generation UI); `InvestigationCopilot` (persistent Q&A, `POST /lookup/{id}/analysis/copilot`); `AiComparisonPanel` (re-run the assessment with a different AI backend and list every historical assessment; only on `/lookup/new`, not `/lookup/[id]`); `ShowReceiptsLink` (a shared control, not a panel -- clicking it calls the `onShowReceipts` callback threaded down from the page, which scrolls/highlights rows in `EvidencePanel`).

Chrome and navigation: `TopSearchBar` (pivot search box); `WorkspaceNav` (links to Basket/Cases/Providers, plus a conditionally-rendered Administration link -- it fetches `getCurrentUser()` in the same `useEffect` that already loaded the live basket count, and shows the link only when `role === "admin"`; this is a UX nicety, not the enforcement, since every `/api/v1/admin/*` route re-checks the role itself); `AiQuickSwitch` (reads/writes the platform-wide active AI backend via `GET`/`POST /api/v1/runtime/ai-active`, affecting the *next* investigation anywhere, no restart); `ExportMenu` (four export buttons, all four working today -- see below); `AskAiPanel` (builds a copy-paste prompt for an external AI chat tool via `lib/aiPrompt.ts`'s `buildAiAnalysisPrompt`, opened in a popup window since a true `<iframe>` embed is blocked by `X-Frame-Options`); `InvestigationActions` ("Add to Basket"/"Add to Case" quick actions); `NetworkAccessPanel` (LAN URL display, `/providers`' Network Access tab).

Provider settings: `ProviderConfigRow` -- one row in the Manage Providers panel, shared by the AI and IOC tabs since both consume the same `RuntimeProviderConfig` shape; expands into an edit form driven by `credential_fields` (IOC providers) or a fixed `api_key` + `model` shape (AI backends).

Administration (`/admin` only): `UsersManagementPanel` -- the Users tab's search/filter/sort/paginate table plus its four dialogs (create, edit name/role, reset password, enable/disable confirmation). Every mutation calls the corresponding `admin.py`-backed function in `lib/api.ts` and re-fetches the page on success; every server-side rejection (last-admin protection, self-role-change, duplicate email) is caught and rendered inline in the dialog that triggered it, not as a page-level error banner.

Security Assessment (`/lookup/[id]` and `/lookup/new`, once completed): `SecurityAssessmentPanel` -- tool-selection buttons (filtered to whichever tools support the current investigation's IOC type), a target-confirmation `Input` the caller must retype to exactly match the seed value, an authorization checkbox, and a findings table with a `Dialog`-based drill-down per finding (severity `Badge`, CVE `Badge`s, and the finding's raw `evidence` JSON pretty-printed). Reuses every primitive the Administration console added (`Input` / `Badge` / `Dialog`) -- no new UI primitives were needed for this panel. One real design gap surfaced and was fixed while building this component: the backend's `POST .../run` accepts one shared `profile` string for every selected tool, but only `nmap` has more than one profile (`quick` / `standard` / `web`) -- every other tool only defines `"standard"`. Rather than change the already-tested backend contract, the panel computes the *intersection* of profile ids valid across every currently-selected tool (`availableProfileIds`, a `useMemo`) and only ever offers those -- in practice this means the profile picker only shows more than one option when `nmap` is the sole selected tool.

`components/ui/button.tsx` and `components/ui/card.tsx` were the only UI primitives until the Administration console needed three more: `input.tsx` (a thin styled wrapper matching the raw `<input>` className every form previously repeated inline), `badge.tsx` (role/status pills -- `default` / `success` / `destructive` / `warning` / `muted` variants), and `dialog.tsx` (the first real wrapper around `@radix-ui/react-dialog`, which was already an installed dependency but had zero call sites anywhere in the codebase before this). All three follow the existing `button.tsx` / `card.tsx` convention exactly: `React.forwardRef`, `cn()`-merged classNames, `class-variance-authority` for variants where relevant. Everywhere outside `/admin`, forms still use raw `<input>` elements and modals still use Radix's `Tabs` / `Progress` / `Tooltip` primitives directly -- the new primitives were added because the Administration console's four dialogs made a shared abstraction worth it, not retrofitted everywhere else.

A gap that's now closed: `ExportMenu`'s "Export PDF" and "Export CSV" buttons call `POST /api/v1/lookup/{lookupId}/export?format=pdf|csv` (`components/dashboard/ExportMenu.tsx`), which is a real, implemented route (`backend/app/api/routes/lookup.py`'s `export_lookup()`, added in an earlier fix pass this engagement, rendering both formats server-side from the same data `GET /{lookup_id}` already returns). All four export buttons work today: "Export Markdown" and "Export JSON" are still generated entirely client-side (`buildMarkdown()` walks the already-loaded `FinalAssessment` object; JSON export just serializes it) and never touch the network, while PDF/CSV are server-rendered.

`frontend/lib/api.ts` -- the single backend client

`lib/api.ts` is the frontend's *only* dependency on the backend contract; no other file constructs a `fetch()` call against `/api/v1/*` directly except the SSE stream reader inside `streamLookup()` itself. Every other page/component imports named functions from this one module.

Resolving the backend URL: `getApiUrl()` (lines 46-51). Every function in this file that calls the backend does so through `getApiUrl()`, not a fixed constant. It resolves in this order: (1) `process.env.NEXT_PUBLIC_API_URL`, an explicit build-time override, if set -- normally left empty; (2) otherwise, if running in the browser (`typeof window !== "undefined"`), `${window.location.protocol}//${window.location.hostname}:${port}`, where `port` is `process.env.NEXT_PUBLIC_BACKEND_PORT` (defaulting to `8000`); (3) a hardcoded `http://localhost:8000` fallback used only during Next.js's server-side render pass, where `window` doesn't exist -- never actually hit for a real request, since every call site in this file fires from a `useEffect` or event handler after mount, not during SSR. Case (2) is what makes one built frontend work identically whether opened via `localhost`, `127.0.0.1`, or another device's LAN URL: the browser's own address bar tells the app which host to call back, so there is no LAN-IP detection or configuration needed on the frontend side at all -- that problem is solved once, in the browser, rather than at build or install time. `ExportMenu.tsx` imports this same function rather than keeping its own copy (it originally had an independent, identically-hardcoded `API_URL` constant -- found and fixed alongside this change, since fixing only `lib/api.ts` would have left exports silently broken from any host other than the one that built the image).

Token storage and auth headers. Access and refresh tokens are stored as plain strings in `localStorage` under the keys `access_token` and `refresh_token` (`getAccessToken()`, lines 53-55). `authHeaders()` (lines 57-60) returns `{ Authorization: "Bearer <token>" }` when a token is present, or `{}` otherwise -- there is no cookie-based session and no `httpOnly` storage; tokens are readable by any JavaScript running on the page.

Login / register / refresh. `login(email, password)` posts to `/api/v1/auth/login` and writes both returned tokens to `localStorage` on success (lines 62-72). `register()` posts to `/api/v1/auth/register`, surfacing the backend's `detail` message (e.g. "Email already registered") on failure (lines 75-86). `refreshAccessToken()` posts the stored refresh token to `/api/v1/auth/refresh`; on any non-`ok` response it calls `logout()` (clearing both tokens) and returns `false` so the caller can redirect to `/login` instead of retrying indefinitely (lines 93-110). `isLoggedIn()` only checks the mere *presence* of an access token -- an expired token still passes it and only fails once an actual request 401s.

`authedFetch()` -- the 401-retry wrapper (lines 128-138). Every authenticated JSON call in this file goes through `authedFetch(url, init)`, which performs the request once with the current access token, and, only if the response is `401`, calls `refreshAccessToken()` and retries the same request exactly once with the new token. If the refresh itself fails, the original `401` response is returned to the caller unmodified (there is no further retry or redirect logic inside `authedFetch()` itself -- that is left to each call site / page, e.g. `streamLookup()`'s own `onAuthExpired` handler described below). This wrapper is what makes a 30-minute-old access token (the backend's `ACCESS_TOKEN_EXPIRE_MINUTES` default) transparently refresh mid-session rather than surfacing a raw 401 to the user, per the function's own inline comment.

Every other exported function is a thin `authedFetch()`-based wrapper around one backend endpoint, generally following the pattern `fetch -> check res.ok -> throw Error(...) or return res.json()`. The full mapping (function name -> backend route) is:

`lib/api.ts` function	Backend route
`getLookup`, `listLookups`	`GET /lookup/{id}`, `GET /lookup`
`getProviderHealth`	`GET /providers/health`
`getEvidence`, `explainWhyMalicious`, `explainWhatIsThis`, `explainDisagreement`, `checkFalsePositive`, `challengeVerdict`, `getNextActions`, `getIntelligenceGaps`, `explainScore`, `askCopilot`	`GET /lookup/{id}/analysis/evidence`, and `POST /lookup/{id}/analysis/{why,what-is-this,disagreement,false-positive,challenge,next-actions,gaps,score-explanation,copilot}` respectively, all funneled through a shared `postAnalysis<T>(lookupId, path, body)` helper (lines 273-281)
`huntThisIOC`, `createDetectionRule`	`POST /lookup/{id}/hunt`, `POST /lookup/{id}/detection`
`getPivots`	`GET /lookup/{id}/pivots`
`listBasket`, `addToBasket`, `removeFromBasket`, `clearBasket`, `compareBasketIOCs`	`GET/POST/DELETE /basket`, `DELETE /basket/{id}`, `POST /basket/compare`
`listCases`, `getCase`, `createCase`, `updateCase`, `addCaseIOC`, `addCaseNote`	`GET/POST /cases`, `GET/PATCH /cases/{id}`, `POST /cases/{id}/iocs`, `POST /cases/{id}/notes`
`listAIProviders`, `configureAIProvider`, `setActiveAIBackend`, `getActiveAIBackend`, `recordAITestResult`, `testAIBackend`	`GET /runtime/ai-providers`, `POST /runtime/ai-providers/{backend}`, `POST / GET /runtime/ai-active`, `POST /runtime/ai-providers/{backend}/record-test`, `POST /ai/test`
`listIOCProviders`, `configureIOCProvider`, `setIOCProviderEnabled`, `recordIOCTestResult`, `testIOCProvider`	`GET /runtime/ioc-providers`, `POST /runtime/ioc-providers/{id}`, `POST /runtime/ioc-providers/{id}/enabled`, `POST /runtime/ioc-providers/{id}/record-test`, `POST /providers/{id}/test`
`getAuditLog`	`GET /runtime/audit-log`
`reanalyzeLookup`, `listAssessments`	`POST /lookup/{id}/reanalyze`, `GET /lookup/{id}/assessments`
`getNetworkInfo`	`GET /network-info` -- unauthenticated (plain `fetch`, not `authedFetch`), same trust model as the backend's own `/health`; powers the Network Access panel's LAN URL display
`getCurrentUser`	`GET /auth/me` -- used by `WorkspaceNav` (role check for the Administration link) and `/admin` (route guard + the "signed in as" line and self-role-change UI hint)
`listUsers`, `getUserStats`, `getRoles`, `createUser`, `updateUser`, `setUserActive`, `resetUserPassword`	`GET /admin/users`, `GET /admin/users/stats`, `GET /admin/roles`, `POST /admin/users`, `PATCH /admin/users/{id}`, `POST /admin/users/{id}/active`, `POST /admin/users/{id}/reset-password` -- the Administration console's entire backend surface
`getSecurityAssessmentProfiles`, `getSecurityAssessmentToolHealth`, `runSecurityAssessment`, `listSecurityAssessmentRuns`, `getSecurityAssessmentRun`	`GET /security-assessment/profiles`, `GET /security-assessment/tool-health`, `POST /security-assessment/{id}/run`, `GET /security-assessment/{id}/runs`, `GET /security-assessment/runs/{run_id}` -- the Security Assessment Toolkit's entire backend surface

Every route in the backend's route inventory now has a corresponding frontend function except `POST /auth/register` and `POST /auth/refresh` (both called directly by name -- `register()`, `refreshAccessToken()` -- rather than through this generic table's naming pattern, since they're described in the Login/register/refresh paragraph above) -- no dead frontend code calls a route the backend lacks.

`frontend/lib/types.ts` -- the Pydantic mirror

`lib/types.ts` is a single, manually-maintained file of `interface` / `type` declarations, explicitly commented at its own top: *"Mirrors `backend/app/providers/base.py` `ProviderResult.to_dict()` and `backend/app/ai/schemas.py` -- keep these in sync when either side changes."* There is no code generation step (no OpenAPI client generator, no shared schema package) -- if a backend Pydantic model's field changes, a developer must hand-edit this file to match, and nothing will fail at build time if they forget (TypeScript will only catch a *usage* of a since-removed/renamed field, not a silently-stale extra or missing one).

Structurally the file groups types by the backend module they mirror, matching its own inline comment headers: provider/AI core (`ProviderStatus` -- an 8-value union matching `app/providers/base.py`; `ProviderResult`, `ProviderSummary`, `MitreMapping`, `DetectionRule`, `RiskAssessment`, `FinalAssessment` mirroring `app/ai/schemas.py`, including the `ai_backend` / `ai_model` traceability fields); runtime config (`RuntimeProviderConfig` mirroring `runtime_config.py`'s `_row_to_public_dict()`, `AuditLogEntry`, `AssessmentRecord`); correlation graph (`GraphNode`, `GraphEdge`, `CorrelationPayload`); lookup listing (`LookupSummary`); evidence/analysis, mirroring `app/models/evidence.py` + `app/ai/analysis_schemas.py` (`EvidenceItem`, `WhyMaliciousExplanation`, `WhatIsThisIOC`, `DisagreementSummary`, `FalsePositiveAssessment`, `ChallengeVerdict`, `SmartNextActions`, `IntelligenceGaps`, `ScoreExplanation`, `CopilotAnswer`, and the basket-compare `IOCComparisonResponse`); hunting/detection (`HuntingPackage`, `DetectionRuleDraft`); pivot (`PivotSuggestion`); basket (`BasketItem`); and case management (`CaseStatus`, `CaseSeverity`, `CaseSummary`, `CaseDetail` and its nested entry types).

Every `Literal`-equivalent backend enum (`Verdict`, `CaseStatus`, `CaseSeverity`, provider `status`, AI `threat_level` / `confidence`) is reproduced as a TypeScript string-union rather than an arbitrary `string`, so the compiler catches an obviously wrong literal even though it cannot catch a genuinely stale field list. A few fields carry inline comments explaining a backend nuance otherwise non-obvious from the type alone -- e.g. `FinalAssessment.ai_backend` / `ai_model` are documented as "both null when no AI was called at all (the no-evidence short-circuit in `ai/service.py` never invokes a backend)."

The SSE-driven live investigation stream, end to end

The single most important piece of client logic in the frontend is `streamLookup()` (`lib/api.ts`, lines 171-251), which consumes the backend's `POST /api/v1/lookup/stream` response. It is deliberately **not** built on the browser's `EventSource` API -- the function's own comment states why: *"EventSource can't send an Authorization header and this endpoint requires a bearer token."* Instead it uses a raw `fetch()` plus a manual `ReadableStream` reader. Like every other function in this file, the URL it POSTs to is built via `getApiUrl()`, so a live investigation streams correctly whether the page was opened locally or from another device's LAN URL.

1. Opening the connection. `streamLookup(value, handlers, signal?, options?)` POSTs `{ value, provider_ids: options?.providerIds, ai_backend: options?.aiBackend }` to `/api/v1/lookup/stream` with the current `Authorization` header attached (lines 171-189). `providerIds` restricts the investigation to specific providers; `aiBackend` pins a specific AI backend for just this one investigation. Neither is exercised by the current `/lookup/new` page (it calls `streamLookup(rawValue.trim(), { ...handlers }, controller.signal)` with no `options`), so both exist in the client library but are not yet wired to any UI control.

2. The auth-retry path is special-cased, not delegated to `authedFetch()`. A streaming `fetch()` response can't be transparently "retried" the way a JSON response can (the body may already be partially consumed), so `streamLookup()` reimplements the 401-then-refresh-then-retry logic inline (lines 190-198). If the retried POST still fails (refresh itself failed), it invokes

`handlers.onAuthExpired?.()` plus a generic `onError` ("Your session expired. Please sign in again.") and returns without throwing.

3. Reading and framing the stream. Once a `200` with a body is confirmed, the function grabs `res.body.getReader()` and a `TextDecoder`, then loops `reader.read()` until `done` (lines 205-250). Decoded bytes accumulate in a `buffer` string that is split on `"\n\n"` (the SSE record separator) each iteration, with the remainder held back for the next chunk -- correctly handling an event split across two stream chunks. Each complete record is split into lines; the function locates the `"event: "` and `"data: "` lines and `JSON.parse()`s the payload.

4. Dispatch table. The parsed `eventName` is switched against exactly the six (plus error) event names the backend's `stream_lookup` handler emits, each routed to an optional handler callback supplied by the caller:

SSE event name	Handler callback	Payload shape (from `lib/types.ts`)
`detected`	`onDetected`	`{ lookup_id, ioc_value, ioc_type }`
`provider_result`	`onProviderResult`	`ProviderResult`
`provider_summary`	`onProviderSummary`	`ProviderSummary`
`correlation`	`onCorrelation`	`CorrelationPayload` (`{ nodes, edges }`)
`final_assessment`	`onFinalAssessment`	`FinalAssessment`
`done`	`onDone`	`{ lookup_id }`
`error`	`onError`	`{ message }`

Any event name not in this list is silently ignored (the `switch` has no `default` case) -- forward-compatible if the backend ever adds a new event type, at the cost of that new event being invisible to old frontend builds until updated.

5. Consumption by the page. `app/lookup/new/page.tsx` calls `streamLookup()` once inside a `useEffect` keyed on `rawValue`, passing an `AbortController.signal` so unmounting (including the remount-on-new-value behavior of `LookupNewPageKeyed`) aborts the in-flight fetch rather than leaking it. Each handler updates local React state (`providerResults`, `providerSummaries`, `correlation`, `finalAssessment`, each a plain `useState`) and appends a line to an `eventLog` array rendered in a sidebar debug panel. Because `provider_result` / `provider_summary` events arrive as a stream of *individual* provider completions (the backend fans out to ~18 providers concurrently, emitting each one's event as it finishes), the handlers merge into a `Record<string, ProviderResult>` / `Record<string, ProviderSummary>` keyed by `provider_id` rather than replacing an array, so the dashboard fills in card by card. `isDone` (set on `done`) gates the post-completion analysis panels (`VerdictAnalysisPanel`, `EvidencePanel`, `PivotPanel`, `HuntingCenterPanel`, `InvestigationCopilot`, `AiComparisonPanel`), which all require a `COMPLETED` lookup.

6. Divergence between the live and persisted views. `app/lookup/[id]/page.tsx` deliberately renders the identical dashboard component tree, but sources its data from one `getLookup(id)` call instead of the SSE stream, and its own header comment documents the exact shape differences it must paper over: the persisted `GET /lookup/{id}` response's `provider_results` rows omit `ioc_value`, `ioc_type`, `fetched_at`, and `from_cache` (fields only meaningful on the live per-call `ProviderResult`), so the page backfills them from the parent lookup or sane defaults (`fetched_at: 0`, `from_cache: false`) before handing the data to the same `ProviderCardGrid` / `ProviderCard` components the live page uses.

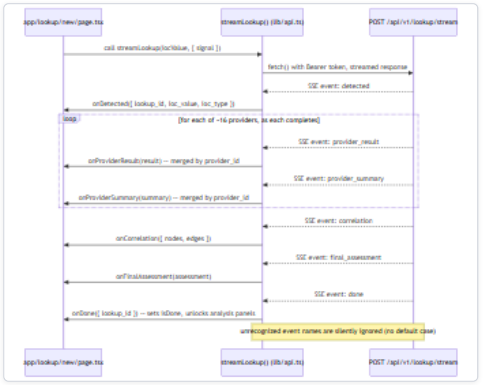

Figure 2 — Diagram: The SSE-driven live investigation stream, end to end

Diagram: SSE event flow from `POST /api/v1/lookup/stream` through `streamLookup()` to dashboard state. Provider events arrive as an incremental stream, one pair per completed provider, rather than a single batch -- which is why the dashboard fills in card-by-card instead of all at once.

Summary

The frontend has exactly one seam to the backend (`lib/api.ts`) and exactly one place where the two sides' data contracts are declared to match by convention rather than by tooling (`lib/types.ts`) -- both facts a maintainer should internalize before changing either side. The live investigation experience is built entirely on a hand-rolled SSE consumer (`streamLookup()`) chosen specifically because the standard `EventSource` API cannot carry the bearer token this platform's every endpoint requires; that one design constraint explains the shape of the entire `/lookup/new` data-loading path. One verified rough edge is worth remembering during future work: `streamLookup()`'s `providerIds` / `aiBackend` per-investigation overrides are implemented in the client library but not yet exposed by any UI control. The Administration console (`/admin`) is this codebase's first screen gated by anything beyond "logged in or not" -- its role check is intentionally UX-only (redirect a non-admin away from a screen full of buttons that would 403), since the actual boundary is server-side on every `/api/v1/admin/*` route, the same "don't trust the frontend for authorization" posture the rest of the backend already followed.

Backend Architecture and Request Flow

This chapter is a code-level reference for how the FastAPI backend is assembled and how one real request moves through it. It covers three things, in order: (1) how `backend/app/main.py` builds the application object — routers, middleware, and startup events; (2) the layering convention the codebase follows (`routes/` → `services/` -equivalent modules → `models/`); and (3) a full, file-and-line trace of `POST /api/v1/lookup/stream` , the platform's single most important endpoint, from the HTTP request to the closing SSE event. Every claim below is anchored to a specific file and, where useful, a line number, as of this codebase.

1. Application assembly (`app/main.py`)

The entire FastAPI app is built in one 65-line module (`app/main.py`). There is no application factory function and no per-environment app-building logic — `app` is a module-level object constructed at import time, from `settings = get_settings()` (`main.py:16`), the `@lru_cache` -decorated `Settings` singleton (`app/core/config.py:13,110-111`). `settings.api_v1_prefix` (`config.py:20` , value `"/api/v1"`) is baked into the OpenAPI URL (`main.py:24`) and every router prefix below at import time, not re-read per request.

Middleware

Exactly one middleware is registered — CORS:

```
# app/main.py:28-34
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"] if settings.debug else [],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Worth flagging precisely for a backend/DevOps audience: when `settings.debug` is falsy (the production default), `allow_origins` evaluates to an **empty list**, not a wildcard and not the production frontend's real origin — no environment variable is read here to populate a production allow-list. In practice, browser-based cross-origin calls to the API in a non-debug deployment have no CORS allow-listed origin configured in this file. No other CORS, CSRF, or security-headers middleware is registered anywhere in the request pipeline.

The only other cross-cutting instrumentation is Prometheus metrics, attached immediately after CORS: `Instrumentator().instrument(app).expose(app, endpoint="/metrics")` (`main.py:36`). This exposes `GET /metrics` with no auth dependency, no `/api/v1` prefix, and no permission check.

Router registration

All ten route modules are imported in one line and registered in a fixed order, each mounted under the same global prefix:

```
# app/main.py:9, 38-47
from app.api.routes import ai_config, analysis, auth, basket, cases, hunting, lookup, pivot, providers, runtime
...
app.include_router(auth.router, prefix=settings.api_v1_prefix)
app.include_router(lookup.router, prefix=settings.api_v1_prefix)
app.include_router(providers.router, prefix=settings.api_v1_prefix)
app.include_router(ai_config.router, prefix=settings.api_v1_prefix)
app.include_router(analysis.router, prefix=settings.api_v1_prefix)
app.include_router(hunting.router, prefix=settings.api_v1_prefix)
app.include_router(pivot.router, prefix=settings.api_v1_prefix)
app.include_router(basket.router, prefix=settings.api_v1_prefix)
app.include_router(cases.router, prefix=settings.api_v1_prefix)
app.include_router(runtime.router, prefix=settings.api_v1_prefix)
```

Each router carries its own sub-prefix declared where it's defined (e.g. `router = APIRouter(prefix="/lookup", tags=["lookup"])` , `app/api/routes/lookup.py:44`), so the final path is always `{api_v1_prefix}{router_prefix}{route_path}` — e.g. `/api/v1` + `/lookup` + `/stream` = `/api/v1/lookup/stream` . Registration order has no effect on route matching (FastAPI matches by path, and no paths overlap across these ten modules); it is simply the order this documentation set uses when enumerating endpoints.

Startup event

One `@app.on_event("startup")` hook exists, `_seed_runtime_config()` (`main.py:50-60`), which calls `seed_from_env_if_empty()` (`app/core/runtime_config.py:398-448`) inside a `try/except Exception` that logs but never blocks boot on failure. That function is the bridge described in the credential-lifecycle chapter of this documentation set: if `provider_runtime_configs` already has any row, it no-ops (`runtime_config.py:408-410`); otherwise it reads every AI-backend/IOC-provider credential out of the frozen `Settings` singleton and inserts one encrypted row per provider/backend — a one-time migration from `.env` -driven to DB-driven configuration. There is no corresponding shutdown hook anywhere in `main.py` .

Two endpoints outside the versioned API

`GET /health` (`main.py:63-65`) and `GET /metrics` (`main.py:36`) are the only two HTTP-reachable endpoints defined directly in `main.py` rather than in `app/api/routes/*.py` . Neither carries the `/api/v1` prefix, and neither has a `require_permission(...)` or `get_current_user` dependency — both are intentionally unauthenticated, `/health` for container liveness checks and `/metrics` for Prometheus scraping.

2. The layering convention: routes → domain services → models

The codebase does not use a single `services/` package name; instead each concern has its own top-level `app/` package, and the convention is consistent regardless of package name: **route handlers in `app/api/routes/*.py` are thin** — they perform auth/permission checks, load/validate the request, and delegate to a domain module, then serialize that module's return value back into the response. Business logic does not live in the route file itself beyond that orchestration.

Layer	Packages	Responsibility
Routes	`app/api/routes/*.py`	HTTP concerns only: request parsing (Pydantic schemas from `app/schemas/`), `Depends(require_permission(...))` / `Depends(get_current_user)` auth gates, calling into the layer below, shaping the JSON/SSE response, raising `HTTPException` for the handful of route-level error cases documented per-endpoint.
Domain/"service" modules	`app/ai/service.py`, `app/ai/analysis_service.py`, `app/ai/hunting_service.py`, `app/providers/orchestrator.py`, `app/correlation/engine.py`, `app/evidence/builder.py`, `app/evidence/loaders.py`, `app/evidence/pivot.py`, `app/core/runtime_config.py`, `app/core/users.py`, `app/core/audit.py`	The actual business logic: AI prompting/backend resolution, provider fan-out/caching/retry, correlation-graph construction, deterministic evidence extraction, runtime-config CRUD, and user-account management (create/edit/enable-disable/reset-password, last-admin protection). These modules are framework-agnostic — none of them import FastAPI, and several (`correlation/engine.py`, `evidence/builder.py`, `evidence/pivot.py`) are explicitly documented in their own module docstrings as pure, I/O-free functions kept that way specifically so they are unit-testable in isolation from the web layer. `app/core/audit.py` is the shared append-only audit-log writer both `runtime_config.py` and `users.py` call into — extracted from `runtime_config.py` (which re-exports both functions unchanged) specifically so a user-management module didn't need to import an entire provider-configuration service module just to write an audit row.
Models	`app/models/*.py`	SQLAlchemy 2.0 ORM entities (`IOCLookup`, `ProviderResultRecord`, `AISummaryRecord`, `CorrelationEdgeRecord`, `FinalAssessmentRecord`, `EvidenceItem`, `User`, `BasketItem`, `Case` / `CaseIOC` / `CaseNote` / `CaseReport`, `ProviderRuntimeConfig` / `ConfigAuditLog`) plus a handful of enums colocated with the model that owns them (e.g. `LookupStatus` / `Verdict` in `app/models/lookup.py`). No business logic lives on these classes beyond SQLAlchemy relationship/mixin declarations (`app/models/base.py`'s `UUIDPrimaryKeyMixin` / `TimestampMixin`).

Two supporting packages sit underneath all three layers, imported by nearly everything above them without depending on any of it: `app/core/` (settings, DB session factory, Redis cache/rate-limiter, Fernet crypto, the per-investigation `ContextVar` credential snapshot) and the Pydantic schema modules (`app/ai/schemas.py`, `app/ai/analysis_schemas.py`, `app/schemas/*.py`) that both routes and domain modules validate against. `app/auth/rbac.py` is the one cross-cutting dependency every protected route imports directly rather than routing through a domain module, since permission-checking is itself an HTTP-layer concern.

Worth calling out for anyone extending the codebase: `app/api/routes/lookup.py` is the one route module that does **not** fully delegate to a single domain module for its core endpoint. `stream_lookup()` (traced in full below) directly orchestrates calls across four domain modules in sequence (`providers/orchestrator.py`, `ai/service.py`, `correlation/engine.py`, `evidence/builder.py`) plus raw SQLAlchemy session writes, inline in the route function, rather than through one composed "run a lookup" service function — a consequence of the SSE streaming requirement, since each domain call's result must be persisted and yielded before the next one starts. This makes it the least "thin" handler in the codebase, and the natural place to read to understand the whole pipeline.

3. Full trace: `POST /api/v1/lookup/stream`

This section follows one request through every layer, in the exact order the code executes it. The route is defined in `app/api/routes/lookup.py:51-303` (`stream_lookup`), mounted at `/api/v1/lookup/stream` (prefix composition described in §1).

Step 0 — Dependency resolution (before the handler body runs)

FastAPI resolves two `Depends(...)` parameters before `stream_lookup`'s body executes (`lookup.py:52-56`):

- `db: AsyncSession = Depends(get_db)` — opens a new `AsyncSession` from the process-wide `async_sessionmaker` (`app/core/db.py:8-14`), used only for the initial `IOCLookup` insert; it is explicitly **not** reused for the rest of the request (see Step 2).
- `user: CurrentUser = Depends(require_permission("lookup:create"))` — depends on `get_current_user` (`app/auth/rbac.py:28-47`), which extracts a Bearer token via `HTTPBearer(auto_error=False)` (`rbac.py:18`), decodes it with `decode_token()`, rejects it with `401 "Could not validate credentials"` if missing/undecodable/not an access token (`rbac.py:32-41`), then looks the user up by the token's `sub` (email) claim and rejects inactive/missing users the same way (`rbac.py:43-46`). `require_permission("lookup:create")` (`rbac.py:50-62`) then checks `"lookup:create" in ROLE_PERMISSIONS.get(user.role, set())` (`app/models/user.py`) and raises `403` if the caller's role lacks it — only `admin` and `analyst` carry `lookup:create`; `viewer` does not.

Step 1 — Rate limiting

```
# lookup.py:61-75
settings = get_settings()
limiter = RateLimiter(
    f"lookup_create:{user.id}",
    max_calls=settings.lookup_rate_limit_max_calls,
    window_seconds=settings.lookup_rate_limit_window_seconds,
)
if not await limiter.allow():
    raise HTTPException(status_code=429, detail=...)
```

`RateLimiter` (`app/core/cache.py:42-56`) is a fixed-window counter stored in Redis under key `rate_limit:lookup_create:{user_id}` (`cache.py:47`): `INCR` the key, and if this is the first increment in the window, set its TTL to `window_seconds` (`cache.py:53-55`). Defaults are `lookup_rate_limit_max_calls = 10` and `lookup_rate_limit_window_seconds = 60` (`app/core/config.py:106-107`) — 10 lookups per 60 seconds, per authenticated user, enforced centrally in Redis so it holds across every backend worker process, not just the one that happens to handle a given request.

Step 2 — IOC type detection and lookup row creation

```
# lookup.py:77-88
ioc_value = payload.value.strip()
ioc_type = payload.ioc_type_hint or detect_ioc_type(ioc_value)
if ioc_type == IOCType.UNKNOWN:
    raise HTTPException(status_code=422, detail="Could not determine IOC type; pass ioc_type_hint.")

lookup = IOCLookup(
    ioc_value=ioc_value, ioc_type=ioc_type.value, status=LookupStatus.RUNNING, requested_by=user.id
)
db.add(lookup)
await db.commit()
await db.refresh(lookup)
lookup_id = lookup.id
```

`detect_ioc_type()` (`app/ioc/detector.py`) runs an ordered regex/heuristic cascade against the raw string to classify it into one of the `IOCType` enum values (`app/ioc/types.py`) — an explicit client-supplied `ioc_type_hint` on the request body takes precedence and skips detection. The `IOCLookup` row (`app/models/lookup.py:37-66`) is inserted and committed using the request-scoped `db` session **before** the SSE generator below is even constructed, so the row (with `RUNNING` status) exists in Postgres before the first byte of the response body is sent.

A load-bearing implementation detail is documented in-line at `lookup.py:93-100` : the `Depends(get_db)` session is torn down as soon as the route function returns the `StreamingResponse` , because generators are lazy — that teardown happens *before* the generator body below ever runs. Using `db` inside the generator would therefore silently drop every later `lookup.status = ...` mutation once the session closes. The fix is that the generator opens and owns a brand-new, independent session (`stream_db` , via `app/core/db.py:17-22` 's `new_session()`) for its entire run. From here forward — every provider result, AI summary, correlation edge, the final assessment, and the evidence ledger — is persisted through `stream_db` , never through the original `db` .

Step 3 — The SSE generator opens and emits `detected`

```
# lookup.py:90-101
async def event_stream():
    yield _sse("detected", {"lookup_id": str(lookup_id), "ioc_value": ioc_value, "ioc_type": ioc_type.value})
    async with new_session() as stream_db:
        stream_lookup_row = await stream_db.get(IOCLookup, lookup_id)
        ...
```

`_sse()` (`lookup.py:47-48`) formats one SSE frame: `f"event: {event}\ndata: {json.dumps(data, default=str)}\n\n"` . The route returns immediately after defining this generator — `return StreamingResponse(event_stream(), media_type="text/event-stream")` (`lookup.py:303`) — so all pipeline work below happens lazily, as the ASGI server pulls values out of `event_stream()` .

Step 4 — Provider fan-out (`detected` → N × `provider_result` / `provider_summary`)

The core loop (`lookup.py:104-150` , abridged):

```
async for result in run_all_providers(ioc_value, ioc_type, candidate_providers):
    provider_results.append(result)
    stream_db.add(ProviderResultRecord(lookup_id=lookup_id, provider_id=result.provider_id, ...))
    if result.status.value == "ok":
        summary = await summarize_provider(ioc_value, ioc_type.value, result, backend_override=payload.ai_backend)
        stream_db.add(AISummaryRecord(lookup_id=lookup_id, provider_id=result.provider_id, summary=summary.model_dump()))
    await stream_db.commit()
    yield _sse("provider_result", result.to_dict())
    if result.status.value == "ok":
        yield _sse("provider_summary", summary.model_dump())
```

`candidate_providers` is `None` (all applicable providers) unless the request body's `provider_ids` narrows it to a client-selected subset (`lookup.py:104-107`). `run_all_providers()` (`app/providers/orchestrator.py:89-127`) is an `async` generator, not a batch call: it filters the 18 registered providers (`app/providers/registry.py`) to those whose `supports(ioc_type)` is true (`orchestrator.py:99-103`); takes **one** snapshot of every IOC provider's enabled/configured/credentials state (`get_ioc_provider_snapshot()` , `app/core/runtime_config.py:297-315` , a fresh DB read) and installs it into a `contextvars.ContextVar` via `set_provider_overrides()` (`app/core/runtime_context.py`), so every concurrently-spawned provider task sees identical configuration for this one investigation even if an admin changes a credential mid-run (`orchestrator.py:105-113`); then opens one shared, connection-pooled `httpx.AsyncClient` and spawns one `asyncio.create_task` per provider via `_run_with_policy()` (`orchestrator.py:115-122`). That function checks the Redis result cache first (`app/core/cache.py:29-31` , key `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` , default TTL 3600s), and on a miss calls `provider.run()` under a `tenacity` retry (only `httpx.ConnectError` / `ReadTimeout` / `PoolTimeout` , default 2 retries) plus an `asyncio.wait_for` timeout (default 20s; `orchestrator.py:29-86` , `config.py:101-103`). Results are yielded via `asyncio.as_completed(tasks)` (`orchestrator.py:123-127`) — completion order, not registration order — which is why the route can stream each `provider_result` the instant that provider finishes.

For each `ok` -status result, the route calls `summarize_provider()` (`app/ai/service.py:336-374`): one AI call grounded only in that provider's own JSON, pruned to a bounded per-field size first (`_prune_for_prompt()` , `service.py:273-325` — a long `str` field is capped at 2000 chars, a long `list` / `dict` field at 800, so a provider's oversized structural data can't bury its own short verdict/score fields under noise), using whichever backend `_get_ai_client()` resolves (explicit request override → active runtime-configured backend, fresh DB read → legacy `Settings.ai_backend` ; `service.py:104-145`). A failed/malformed AI call degrades to a placeholder `ProviderSummary` rather than raising (`service.py:365-374`) — one provider's summarization failure never aborts the request.

Every iteration ends with `await stream_db.commit()` (`lookup.py:147`) — a commit **per provider**, not one at the end of the loop. Per the source comments, this was added after confirming live that a client disconnecting mid-investigation discarded every uncommitted `ProviderResultRecord` / `AISummaryRecord` : the lookup correctly flipped to `FAILED` , but `GET /lookup/{id}` came back with an empty `provider_results` list despite several real provider calls having already completed and streamed (`lookup.py:133-146`). Committing per-provider bounds that loss to at most the one result in flight at disconnect time.

Step 5 — Correlation (`correlation` event)

`correlate(ioc_value, ioc_type, provider_results)` (`lookup.py:152` ; `app/correlation/engine.py:97+`) runs once every provider task has resolved. It is a pure, I/O-free function: it walks a fixed table of known provider-response field names (`_RELATIONSHIP_EXTRACTORS` , `engine.py:57-70` — e.g. `resolved_ips` → `resolves_to` , `mitre_techniques` → `uses_technique` , `cves` → `exploits`) to extract typed `GraphEdge` s, assigns each a base confidence per source field (`_FIELD_BASE_CONFIDENCE` , `engine.py:77-90`), and boosts confidence `+0.15` per additional distinct provider independently asserting the identical `(source, target, relationship)` triple (`_CORROBORATION_BONUS_PER_PROVIDER` , `engine.py:94`), capped at `1.0` . The returned `CorrelationResult` (`engine.py:42-47` : `nodes` , `edges` , `deduplicated_facts` , `provider_agreement`) feeds Steps 6-7. Every edge is persisted as a `CorrelationEdgeRecord` (`app/models/lookup.py:100-118` , `lookup.py:153-165`) and the full node/edge set streamed in one `correlation` SSE event (`lookup.py:166-172`).

Step 6 — Final AI assessment (`final_assessment` event)

The route re-reads AI summaries from Postgres rather than reusing the in-memory list from Step 4 (`lookup.py:174-185`) — a consistency choice ensuring the final assessment is grounded in exactly what was persisted. Rows that fail `ProviderSummary.model_validate()` are logged and skipped, not fatal (`lookup.py:179-185`). It also builds an `unavailable_providers` list distinguishing providers that errored/timed out/were rate-limited/disabled/unconfigured from providers that ran and genuinely found nothing (`lookup.py:192-203`) — a distinction `generate_final_assessment()` 's system prompt depends on to avoid describing an unavailable provider's silence as "clean."

`generate_final_assessment()` (`app/ai/service.py:378-540`) makes at most two AI calls (`for attempt in range(2)` , `service.py:488`) — a bounded retry scoped specifically to `pydantic.ValidationError` (`service.py:513`), which `FinalAssessment` 's own model validator raises when the model emits a self-contradictory verdict/probability pair (e.g. `final_verdict="malicious"` with a low `malicious_probability`) — not a generic retry-on-any-failure: any other exception (`service.py:517`) fails fast after one attempt, since retrying a rate-limit or network error immediately would just waste more of the same budget rather than fix anything. Grounded only in the per-provider summaries plus the correlation output, never raw provider JSON. Two more anti-hallucination controls wrap it:

- **A hard-coded empty-evidence short-circuit** (`service.py:334-370`): if both `provider_summaries` and `correlation.edges` are empty, the function returns a deterministic `FinalAssessment` with `final_verdict=Verdict.UNKNOWN` and never calls the AI. The in-line comment documents the incident that motivated this: a real EICAR-test-file hash, with zero configured providers, still produced `final_verdict="highly_malicious"` (`malicious_probability=92`) with a fabricated ransomware/trojan attribution — the model answered from pretrained knowledge of a famous test hash rather than the (empty) supplied evidence, despite being instructed not to. The fix is deterministic code rather than a prompt change, since the model had already been told not to do this and did it anyway.
- **`_ground_final_assessment()`** (`service.py:211-264`), run on every non-empty-evidence response: strips any provider named in `agreeing_providers` / `disagreeing_providers` not backed by a real correlation edge, `provider_agreement` entry, or per-provider summary (`known_provider_ids` is unioned in so providers like the OSINT crawler — which returns data but produces no correlation edge or verdict fact — aren't misclassified as hallucinated); and flags (rather than strips) any `mitre_mappings` entry not backed by a real `uses_technique` edge, by setting its `grounded` field to `False` .

The resolved `ai_backend` / `ai_model` actually used is written back onto the assessment object (`service.py:426-427`), overwriting whatever the model itself may have emitted — this is what makes `POST /lookup/{id}/reanalyze` (AI-comparison re-analysis) trustworthy even though it calls the same function. Back in the route, `IOCLookup` 's `status` / `final_verdict` / `risk_score` / `confidence_score` / `final_assessment` are updated in place, and a `FinalAssessmentRecord` (`app/models/lookup.py:121-144`) is inserted with `is_primary=True` — the row every later comparison run (`is_primary=False`) is diffed against (`lookup.py:214-228`).

Step 7 — Evidence ledger build (no SSE event of its own)

`build_evidence(provider_results, provider_summaries, correlation)` (`lookup.py:235` ; dispatcher at `app/evidence/builder.py:142-147`) concatenates `build_evidence_from_providers()` (one record per provider that returned OK data, plus one per interesting finding, `builder.py:65+`) with `build_evidence_from_correlation()` (one record per correlation edge, typed by target — `malware_association` , `threat_actor_association` , `campaign_association` , `mitre_technique` , `infrastructure` , or generic `relationship` ; `_relationship_evidence_type()` , `builder.py:51-62`). Per the module docstring, this is a deliberately separate, deterministic pass, not folded into the AI call above, "so 'the AI made this up' and 'the AI cited real evidence' are mechanically distinguishable" (`builder.py:1-6`). The resulting `EvidenceItem` rows and the Step 6 `IOCLookup` / `FinalAssessmentRecord` mutations are all flushed in the same `stream_db.commit()` (`lookup.py:236-252`).

Step 8 — Closing events and error/disconnect handling

`yield _sse("final_assessment", ...)` then `yield _sse("done", ...)` close a successful run (`lookup.py:254-255`). Any exception raised anywhere in Steps 4-7 is caught by one `except Exception` block (`lookup.py:256-260`), which marks the lookup `FAILED` , commits, and emits an `error` SSE event instead — the HTTP response itself stays `200 text/event-stream` throughout, since it has already started streaming by the time any pipeline code runs; failures are communicated only via the event stream, never an HTTP error status.

A second, separate failure mode lives in a `finally` block (`lookup.py:261-301`): a client disconnecting mid-stream raises `GeneratorExit` — a `BaseException` , not an `Exception` — at whichever `yield` is executing, so the `except Exception` above never sees it. Left unhandled, this was confirmed in testing to leave the lookup permanently `RUNNING` , with `GET /lookup/{id}` never transitioning to `completed` / `failed` . A first fix attempt (reusing `stream_db` inside a shielded commit) was confirmed *worse* — the enclosing `async with` block's own rollback raced the shielded task, producing an orphaned task and a `ResourceClosedError` (`lookup.py:275-287`). The working fix (`lookup.py:288-301`): if the lookup is still `RUNNING` when `finally` runs, a nested `_mark_failed()` coroutine opens a **fresh** `new_session()` , re-fetches the row, and flips it to `FAILED` , with the whole operation wrapped in `asyncio.shield()` so disconnect-driven task cancellation cannot cut off the cleanup write itself.

Summary of the full path

Step	Code location	I/O	Persists
Auth + permission check	`rbac.py:28-62`	Postgres (user lookup)	—
Rate limit	`lookup.py:61-75` , `cache.py:42-56`	Redis	—
IOC type detection + lookup row	`lookup.py:77-88` , `ioc/detector.py`	Postgres	`ioc_lookups` (status=RUNNING)
Provider fan-out	`orchestrator.py:89-127`	Redis (cache), N × external HTTP	`provider_results` (per provider)
Per-provider AI summary	`ai/service.py:336-374`	1 AI call per OK provider	`ai_summaries` (per provider)
Correlation	`correlation/engine.py:97+`	none (pure function)	`correlation_edges`
Final AI assessment	`ai/service.py:378-540`	up to 2 AI calls (bounded retry on a validation failure)	`ioc_lookups` (final fields), `final_assessment_records`
Evidence build	`evidence/builder.py:142-147`	none (pure function)	`evidence_items`
SSE close	`lookup.py:254-301`	—	`ioc_lookups.status` (COMPLETED/FAILED)

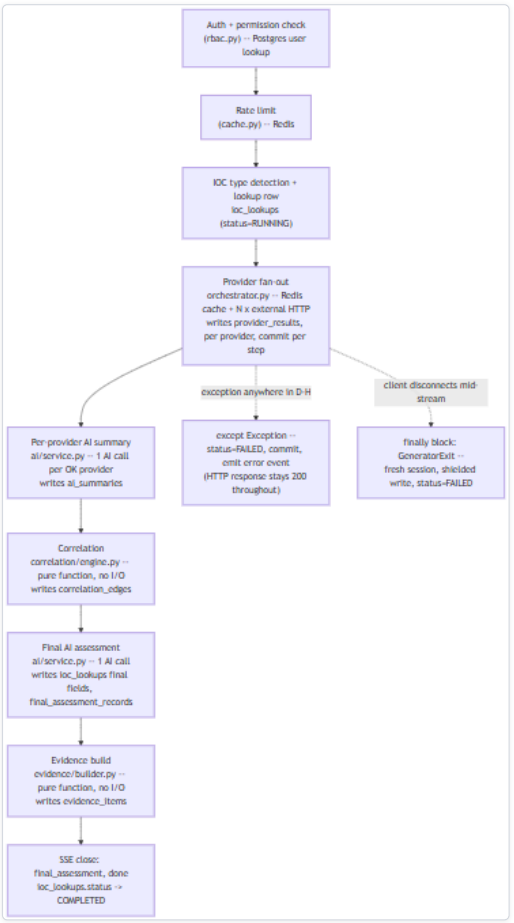

Figure 3 — Diagram: Summary of the full path

Diagram: `POST /api/v1/lookup/stream`, end-to-end sequence. Each stage after provider fan-out commits its own writes independently, so a mid-stream failure or disconnect only loses the one step in flight, never everything already completed.

Every other endpoint in the platform (`analysis.py`'s why/what-is/disagreement/etc., `hunting.py`, `pivot.py`, `basket.py`, `cases.py`) operates on an already-completed lookup's persisted rows and follows the same routes → domain-module → models layering, but without the streaming/session-ownership complexity described above, since none of them return a `StreamingResponse` — they load, call one domain function, and return a single JSON body.

Provider and AI Client Architecture (Developer Reference)

This chapter is the source-level companion to the *Provider Architecture* and *AI Architecture* chapters elsewhere in this documentation set. Those chapters describe what the provider and AI layers *do*; this one documents the actual interfaces, classes, and method signatures a developer extending or maintaining this codebase needs to implement or call. All facts below are drawn directly from `backend/app/providers/base.py`, `registry.py`, `orchestrator.py`, `backend/app/ai/service.py`, and the eleven AI client modules (`ollama_client.py`, `anthropic_client.py`, `bedrock_client.py`, `gemini_client.py`, `groq_client.py`, `openai_client.py`, `kimi_client.py`, `deepseek_client.py`, `xai_client.py`, `mistral_client.py`, `openrouter_client.py`).

1. The `BaseProvider` Contract (`app/providers/base.py`)

Every connector subclasses the abstract class `BaseProvider` (`base.py:80-183`). The module docstring states the intent directly: "the orchestrator never knows about concrete providers — it only calls this interface" (`base.py:1-7`).

Enums. `ProviderCategory` (`base.py:21-28`) has seven values: `THREAT_INTEL`, `SANDBOX`, `PASSIVE_DNS`, `CERTIFICATE_INTEL`, `WHOIS`, `VULNERABILITY`, `OSINT`. `ProviderStatus` (`base.py:31-42`) has eight: `OK`, `ERROR`, `TIMEOUT`, `RATE_LIMITED`, `NOT_CONFIGURED`, `UNSUPPORTED_IOC`, `NO_DATA`, `DISABLED`. The inline comment on `DISABLED` (`base.py:39-41`) explains why it is distinct from `NOT_CONFIGURED`: "a provider can have a perfectly valid key and still be intentionally disabled" — this is the runtime toggle behind `POST /api/v1/runtime/ioc-providers/{id}/enabled`.

`ProviderResult` (`base.py:45-77`) is a `@dataclass`, not a Pydantic model, returned by every provider regardless of vendor response shape:

Field	Type	Notes
`provider_id`, `provider_name`	`str`	e.g. `"virustotal"`
`category`	`ProviderCategory`	
`status`	`ProviderStatus`	
`ioc_value`	`str`	submitted indicator
`ioc_type`	`IOCType`	detected type
`data`	`dict[str, Any]`	default `{}`; the normalized payload
`raw`	`Any`	default `None`; unused by every connector inspected
`source_url`, `error_message`	`Optional[str]`	
`latency_ms`	`Optional[int]`	set by `run()`, not `fetch()`
`fetched_at`	`float`	default `time.time()`
`from_cache`	`bool`	default `False`; set `True` by the orchestrator on a cache hit

`to_dict()` (`base.py:63-77`) serializes the dataclass (enum members → `.value`) for the SSE stream and the Redis cache.

Class shape. A concrete provider sets six class attributes — `provider_id`, `provider_name`, `category`, `supported_types: set[IOCType]`, `requires_key: bool`, `base_url: str` — plus `configured: bool`, and implements exactly one abstract method, `async def fetch(self, ioc_value: str, ioc_type: IOCType, client: httpx.AsyncClient) -> ProviderResult` (`base.py:80-183`). `configured` is a plain attribute set in `__init__` (e.g. `bool(get_settings().virustotal_api_key)`), or hardcoded `True` for key-free providers such as `nvd` or `crtsh`. `supports(ioc_type)` (`base.py:94-95`) is simply `ioc_type in self.supported_types`.

`run()` (`base.py:97-176`) is what the orchestrator calls, never `fetch()` directly. Its docstring disclaims retry/timeout responsibility entirely: that belongs to the orchestrator (§3). `run()`'s own sequence:

1. Reads the per-investigation `ContextVar` override via `get_provider_override(self.provider_id)` (`base.py:104-111`; see §3.2), computing `effective_enabled` / `effective_configured` from it if present, else from the class attributes.
2. If disabled → `ProviderStatus.DISABLED`, `fetch()` never called (`base.py:112-123`).
3. If `supports(ioc_type)` is false → `ProviderStatus.UNSUPPORTED_IOC` (`base.py:124-134`).
4. If `requires_key` and not configured → `ProviderStatus.NOT_CONFIGURED`, `error_message=f"{self.provider_name} is not configured (missing API key/credentials)."` (`base.py:135-146`).
5. Otherwise calls `fetch()` inside `try/except`: an `httpx.HTTPStatusError` maps to `RATE_LIMITED` for status codes `429`, `403`, `509` (509 annotated as "PhishTank's documented over-limit response code", `base.py:152`) and to `ERROR` otherwise; any other exception is caught by a bare `except Exception` and normalized to `ERROR` with `error_message=str(exc)` (`base.py:147-174`).
6. `latency_ms` is stamped from a `time.monotonic()` delta on every path (`base.py:175`).

Because every branch above is centralized, a concrete `fetch()` only needs to handle its own vendor-specific success/404/no-data cases — anything else it lets propagate is normalized by `run()`.

2. The Provider Registry (`app/providers/registry.py`)

The registry is a flat list and two accessor functions, no logic: `_ALL_PROVIDERS: list[BaseProvider]` holds one module-level singleton instance per connector (imported from its own module, never constructed here); `get_all_providers()` returns it; `get_provider_health()` maps it to the dicts backing `GET /api/v1/providers/health` (`registry.py:8-64`). The docstring states the extensibility contract: "adding a new provider is a two-line change (import + append) and never touches the orchestrator, API routes, or correlation engine" (`registry.py:1-7`). Note `get_provider_health()` reports the class-level `configured` flag, not the per-investigation runtime-override value from §3 — worth knowing when comparing this endpoint against what an in-flight investigation actually saw.

2.1 The 18 registered connectors

provider_id	Category	IOCType s supported	requires_key	Credential setting (app/core/config.py)	Base endpoint
virustotal	threat_intel	IPV4, IPV6, DOMAIN, URL, hashes	Yes	virustotal_api_key (header x-apikey)	virustotal.com/api/v3
abuseipdb	threat_intel	IPV4, IPV6	Yes	abuseipdb_api_key (header Key)	api.abuseipdb.com/api/v2/check
otx	threat_intel	IPV4, IPV6, DOMAIN, HOSTNAME, URL, hashes	Yes	otx_api_key (header X-OTX-API-KEY)	otx.alienvault.com/api/v1
urlhaus	threat_intel	URL, DOMAIN, IPV4	Yes	abusech_auth_key (header Auth-Key, shared)	urlhaus-api.abuse.ch/v1
threatfox	threat_intel	IPV4, IPV6, DOMAIN, URL, MD5, SHA256	Yes	abusech_auth_key (shared)	threatfox-api.abuse.ch/api/v1/
malwarebazaar	threat_intel	MD5/SHA1/SHA256/SHA512	Yes	abusech_auth_key (shared)	mb-api.abuse.ch/api/v1/
crtsh	certificate_intel	DOMAIN, TLS_CERTIFICATE	No	—	crt.sh
nvd	vulnerability	CVE	No	optional nvd_api_key (header apiKey)	services.nvd.nist.gov/rest/json/cves/2.0
cisa_kev	vulnerability	CVE	No	— (in-memory, 3600s TTL, lock-guarded)	CISA KEV JSON feed
mitre_attack	threat_intel	MITRE_TECHNIQUE	No	— (STIX bundle, 3600s TTL, lock-guarded)	MITRE cti GitHub raw JSON
whois_rdap	whois	DOMAIN (WHOIS); IPV4/IPV6/ASN (RDAP)	No	—	python-whois ; rdap.org
hybrid_analysis	sandbox	SHA256 only	Yes	hybrid_analysis_api_key (header api-key)	hybrid-analysis.com/api/v2
spamhaus	threat_intel	IPV4, DOMAIN	No	— (raw DNS, no HTTP)	*.zen / *.dbl.spamhaus.org
phishtank	threat_intel	URL	No	optional phishtank_api_key (form app_key)	checkurl.phishtank.com/checkurl/
censys	passive_dns	IPV4, IPV6	Yes (both fields)	Bearer token **and** X-Organization-ID	platform.censys.io/v3/global/asset/host/{ip}
internet_intelligence	osint	DOMAIN, IPV4, MALWARE_FAMILY, THREAT_ACTOR, CAMPAIGN, CVE, FILE_NAME	No	—	crawler/sources/{github,reddit,rss_news,pastebin_search}.py
urlscan	sandbox	URL, DOMAIN	Yes	urlscan_api_key (header API-Key)	urlscan.io/api/v1 (submit-then-poll)
google_safe_browsing	threat_intel	URL, DOMAIN	Yes	google_safe_browsing_api_key (query param key)	safebrowsing.googleapis.com/v4

All 18 dispatch through the identical run() / fetch() contract in §1; this table exists to show where each one's credential and endpoint live for anyone tracing a credential end to end. urlscan and google_safe_browsing are the two exceptions to the setup wizard's provider page (see the Provider Guide) — both require a key but are configured after install via the app's own Providers page instead. The IOCType enum (app/ioc/types.py:5-38) has 33 members including UNKNOWN ; each provider's supported_types set is a subset.

3. The Orchestrator: Concurrent Fan-Out (app/providers/orchestrator.py)

3.1 _run_with_policy() — per-provider cache, timeout, retry

Sequence for async def _run_with_policy(provider, ioc_value, ioc_type, client) -> ProviderResult (orchestrator.py:29-86):

1. **Cache check**: get_cached_result(provider.provider_id, ioc_type.value, ioc_value) against Redis; on a hit, the cached dict is rehydrated into a ProviderResult with from_cache=True and no outbound call is made.
2. **Timeout**: asyncio.wait_for(provider.run(...), timeout=settings.provider_timeout_seconds) .
3. **Retry**: tenacity.AsyncRetrying — stop_after_attempt(settings.provider_max_retries + 1) , wait_exponential(multiplier=0.5, max=4) , retrying only _RETRYABLE_EXC = (httpx.ConnectError, httpx.ReadTimeout, httpx.PoolTimeout) , reraise=True . This retry layer sits above BaseProvider.run() , consistent with run() 's own disclaimer in §1.
4. **Exhaustion normalization**: a caught asyncio.TimeoutError → ProviderStatus.TIMEOUT ; a retryable exception surviving all attempts → ProviderStatus.ERROR with "Connection error after retries: {exc}" .
5. **Cache write**: only on ProviderStatus.OK , set_cached_result(...) writes result.to_dict() with TTL settings.provider_cache_ttl_seconds . Non-OK results, including NO_DATA , are never cached.

3.2 run_all_providers() — the streaming fan-out generator

async def run_all_providers(ioc_value, ioc_type, providers=None) -> AsyncIterator[ProviderResult] (orchestrator.py:89-127):

1. Filters candidates (all registered providers, or an explicit subset — how the provider_ids field on POST /api/v1/lookup/stream scopes one investigation) to applicable = [p for p in candidates if p.supports(ioc_type)] , **before** dispatch — deliberate, per the inline comment, so a UI's total-provider count reflects only providers that could plausibly contribute.

2. Reads one fresh snapshot per investigation — `snapshot = await get_ioc_provider_snapshot()` (`app/core/runtime_config.py`) — then `set_provider_overrides(snapshot)` (`app/core/runtime_context.py`), **before** any task is spawned.
3. Opens one shared `httpx.AsyncClient` for the whole fan-out (`Limits(max_connections=50, max_keepalive_connections=20)` , `follow_redirects=True`).
4. Spawns one `asyncio.create_task(_run_with_policy(p, ...))` per applicable provider, all at once. This is why the step-2 snapshot is concurrency-safe: `asyncio.create_task()` copies the current `contextvars.Context` into each spawned task, so every task launched for *this* investigation sees the identical frozen snapshot regardless of concurrent writes to the runtime-config table.
5. Consumes via `asyncio.as_completed(tasks)` and `yield` s each `ProviderResult` the instant it is ready — not in submission order. A wrapping `try/except Exception` (annotated "last-resort guard, providers already normalize errors") logs and swallows anything that escapes `_run_with_policy` .

`run_all_providers_collected()` (`orchestrator.py:130-136`) is a non-streaming wrapper — `[r async for r in run_all_providers(...)]` — used by the correlation engine and AI service, neither of which can run until every provider has reported back.

Figure 4 — Diagram: 3.2 `run_all_providers()` — the streaming fan-out generator

Diagram: Provider call sequence -- registry to orchestrator to `BaseProvider.run()` to `fetch()` . The cache check and the `ContextVar` credential check both happen before any outbound HTTP call, and results stream back in whatever order each provider actually finishes, not the order tasks were spawned.

4. The AI Client Interface: `call_claude_json`

`app/ai/service.py` defines the required shape as a `typing.Protocol` , not a base class — each of the eleven client modules independently implements a matching method:

```
class _AIClient(Protocol):
    is_configured: bool
    async def call_claude_json(self, system_prompt: str, user_prompt: str,
                               json_schema: dict, tool_name: str = ...,
                               max_tokens: int | None = ...) -> dict: ...
```

(`service.py:43-53`). Because this is structural typing, adding a twelfth backend requires only a class exposing this method name/signature and an `is_configured` property — no shared inheritance, no registration beyond the branch added to `_build_client()` (§5.2).

Client class	Module	Endpoint / call	Auth	Structured-output technique
`OllamaClient`	`ollama_client.py`	`POST {base_url}/api/chat`, default `http://host.docker.internal:11434`	none (local)	`format` = ref-flattened JSON Schema (grammar-constrained decoding)
`AnthropicClient`	`anthropic_client.py`	`POST api.anthropic.com/v1/messages`	header `x-api-key` + `anthropic-version`	forced single tool call; native `$ref` / `$defs`, no flattening
`BedrockClaudeClient`	`bedrock_client.py`	boto3 `bedrock-runtime.converse()` via `asyncio.to_thread`	bearer token or IAM key/secret	forced tool call via `toolConfig`
`GeminiClient`	`gemini_client.py`	`POST {API_BASE}/models/{model_id}:generateContent`	header `x-goog-api-key` (deliberately not `?key=...`, to avoid the key landing in httpx's INFO-level URL logs)	`responseMimeType: application/json` + ref-flattened `responseSchema`
`GroqClient`	`groq_client.py`	`POST api.groq.com/openai/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema
`OpenAIClient`	`openai_client.py`	`POST api.openai.com/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema
`KimiClient`	`kimi_client.py`	`POST api.moonshot.ai/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema; default model pinned to `kimi-k2.5` since several newer Moonshot models' always-on "thinking" mode is incompatible with a forced `tool_choice`
`DeepSeekClient`	`deepseek_client.py`	`POST api.deepseek.com/chat/completions` (no `/v1` segment)	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema
`XAIClient`	`xai_client.py`	`POST api.x.ai/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema (best-effort — xAI's own docs say `parameters` isn't strictly enforced)
`MistralClient`	`mistral_client.py`	`POST api.mistral.ai/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling, ref-flattened schema
`OpenRouterClient`	`openrouter_client.py`	`POST openrouter.ai/api/v1/chat/completions`	header `Authorization: Bearer`	forced tool-calling; model discovery filtered to `tool_choice`-capable ids, since most of the hundreds of underlying models OpenRouter fans out to don't support forced tool-calling at all

Every constructor accepts `Optional` overrides for credentials/model/`max_tokens`, falling back to `get_settings()` when `None` — e.g. `AnthropicClient.__init__(self, api_key=None, model_id=None, max_tokens=None)` (`anthropic_client.py:26-35`) resolves against `settings.anthropic_api_key` / `anthropic_model_id` / `anthropic_max_tokens`. Each module also exposes a lazily-initialized singleton getter (e.g. `get_anthropic_client()`), used only as the no-runtime-config fallback (§5.2). Ollama, Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, and OpenRouter all run their schema through `app/ai/schema_utils.py`'s `inline_refs()` first, since their schema compilers don't reliably resolve `$ref` / `$defs`; Anthropic and Bedrock's Converse API resolve refs natively and skip that step.

5. `app/ai/service.py`: Backend Resolution and the Two-Step Flow

`_model_id_for_backend(backend, settings)` (`service.py:33-40`) is a dict lookup (`ollama` → `settings.ollama_model`, `anthropic` → `anthropic_model_id`, etc.) used only for attribution/logging, not client construction.

`_build_client(backend, credentials, model_id)` (`service.py:56-99`): when `credentials` is `None` (no runtime config seeded yet), returns the legacy module-level singleton; otherwise constructs a **brand-new client instance** from the decrypted, request-scoped credentials — e.g. for Bedrock, `BedrockClaudeClient(bedrock_api_key=credentials.get("bedrock_api_key"), aws_access_key_id=..., aws_secret_access_key=..., aws_region=..., model_id=model_id)`. The docstring states the rationale: a fresh instance per call is what makes switching backends/credentials at runtime take effect on the very next call, with no restart.

`_get_ai_client(backend_override=None) -> tuple[_AIClient, str, Optional[str]]` (`service.py:102-145`) resolves in order: (1) `backend_override` if given — used only by the reanalyze/AI-comparison feature, and explicitly must not change the platform-wide active backend; (2) the active runtime-configured backend via `get_active_ai_config()` — a fresh DB read every call; (3) the legacy `settings.ai_backend` (default `"ollama"`), only if no runtime config exists at all. If the resolved client's `is_configured` is false, it raises immediately: `RuntimeError(f"AI backend '{backend}' is not configured (missing API key/URL/model) -- configure it from the AI Providers panel or check .env")` — a deliberate fail-fast rather than letting a misconfigured client fail deep inside an HTTP call.

`summarize_provider(ioc_value, ioc_type, result, backend_override=None) -> ProviderSummary` (`service.py:336-374`): if `result.status != ProviderStatus.OK` or `result.data` is empty, returns a hardcoded fallback `ProviderSummary` (`reputation="unknown"`, `detection_status="no data"`, `threat_level="none"`, `confidence="low"`) with **no AI call**. Otherwise the provider's raw data is first passed through `_prune_for_prompt()` (`service.py:273-325` — caps any oversized `str` field at 2000 chars, any oversized `list` / `dict` field at 800, so a provider's bulky structural data can't bury its own short verdict/score fields), then calls `client.call_claude_json(system_prompt=_PROVIDER_SUMMARY_SYSTEM_PROMPT, ..., json_schema=ProviderSummary.model_json_schema(), tool_name="emit_provider_summary")` and validates the result. Any exception is caught and degrades to the same fallback shape with a static `caveats` message pointing at the server logs (`service.py:365-374`), not the exception text itself.

`generate_final_assessment(ioc_value, ioc_type, provider_summaries, correlation, backend_override=None, unavailable_providers=None) -> FinalAssessment` (`service.py:378-540`). `unavailable_providers` (`[{"provider_id", "reason"}]`) covers providers applicable to the IOC type that produced nothing usable this run; it is rendered as a "Providers unavailable this run" prompt section with an explicit instruction never to read that absence as "reported clean." Makes at most two AI calls: `FinalAssessment`'s own model validator rejects a self-contradictory verdict/probability pair (e.g. `final_verdict="malicious"` with a low `malicious_probability`), and that specific `ValidationError` (`service.py:513`) gets one retry — a stochastic model's next sample isn't the same sample — while every other exception (`service.py:517`) fails fast after a single attempt, so a rate-limited or network-failed call doesn't get retried into wasting more of the same budget.

No-evidence guard (`service.py:334-370`): if both `provider_summaries` and `correlation.edges` are empty, returns a hardcoded `FinalAssessment` (`final_verdict=Verdict.UNKNOWN`, all risk fields `0`) **without calling the AI backend**. The code comment documents the incident that forced this: querying the EICAR test file's real MD5 (`44d88612fea8a8f36de82e1278abb02f`) with zero usable provider data still made `llama3.2:3b` return `final_verdict="highly_malicious"`, `malicious_probability=92`, fabricating a ransomware/trojan association from pretrained knowledge — a violation of its own "never fabricate" system-prompt rule that no prompt rewording reliably fixed.

When evidence exists, the function builds `summaries_block` (one section per `ProviderSummary`) and `correlation_block` (`provider_agreement`, `deduplicated_facts`, up to the first 50 correlation edges as `source --relationship--> target [provenance]`), calls `client.call_claude_json(..., json_schema=FinalAssessment.model_json_schema(), tool_name="emit_final_assessment", max_tokens=8192)`, validates it, and passes it through `_ground_final_assessment()` (§6). `assessment.ai_backend` / `ai_model` are then **overwritten** with the backend/model actually invoked (`service.py:426-427`) — necessary because nothing stops the model from filling those fields in itself, and because `settings.ai_backend` can disagree with the runtime-active backend or an explicit `backend_override`. On any exception, the function returns a degraded `FinalAssessment` (`final_verdict="unknown"`, exception `repr()` embedded in the summary fields) — this pipeline never raises out to its caller.

6. `_ground_final_assessment()` — Trusting Nothing at Face Value

`_ground_final_assessment(assessment, correlation, known_provider_ids=None) -> FinalAssessment` (`service.py:211-264`) runs on every generated (non-guard-shortcut) result. It builds `real_provider_ids` as the union of: every `provider_id` in a correlation edge's `provenance`, every provider in `correlation.provider_agreement`, and `known_provider_ids` — `{s.provider_id for s in provider_summaries}`, passed in by the caller. The third source matters because a provider like `internet_intelligence` returns only `osint_findings` / `source_count` — fields the correlation engine's relationship-extraction table doesn't recognize — so without it, a correct citation of that provider as agreeing/disagreeing would be indistinguishable from a hallucinated one and get stripped.

`agreeing_providers` / `disagreeing_providers` are each filtered down to IDs in `real_provider_ids`; anything else is **dropped, not reassigned** — the code comment notes that guessing which list a hallucinated name belongs in "could silently misclassify a provider's position — worse than leaving it missing." Separately, every `MitreMapping.grounded` is set `True` only if its `technique_id` matches the target of a real `uses_technique` correlation edge — an ungrounded technique is flagged, not removed, leaving the UI free to render it differently rather than presenting it as fact.

7. Extending the System

New IOC provider: subclass `BaseProvider`, set the six class attributes, implement `fetch()`, instantiate a module-level singleton, add two lines to `registry.py` (import + append). No other file changes — the orchestrator, correlation engine, and evidence builder consume providers exclusively through the `BaseProvider` / `ProviderResult` contract in §1.

New AI backend: implement a class exposing `is_configured` and an async `call_claude_json(system_prompt, user_prompt, json_schema, tool_name="emit_result", max_tokens=None) -> dict` matching §4's Protocol, add a branch to `_build_client()` and an entry to `_model_id_for_backend()`, and add the corresponding `Settings` fields in `app/core/config.py`. No change to `summarize_provider()`, `generate_final_assessment()`, or §6's grounding logic is required — both call sites reach the backend exclusively through `_get_ai_client()`.

Runtime Configuration and Credential Lifecycle

Every IOC provider connector and every AI backend in this platform needs at least one piece of configuration — usually an API key, sometimes a base URL or model id — before it can do real work. This chapter is the authoritative, line-cited account of where that configuration comes from, how it is stored, and how it is read back at the moment an investigation actually runs, for both of the two credential systems that coexist in this codebase today: the original `.env`-only design, and the encrypted, database-backed runtime layer that was later built on top of it without removing it. No git history exists in this repository (`git status` at the repo root returns `fatal: not a git repository`), so the "before" state below is reconstructed from the legacy code paths that are still present and still load-bearing as a fallback, not from commit history.

Two Entry Points for a Credential

A credential can enter the running system in exactly two ways:

Entry point	Destination	Mechanism	Restart required?
Windows Setup Wizard (Welcome → Admin → AI Configuration → Provider Configuration → Ports → Summary → Install)	`.env` file on disk	`Write-PlatformEnvFile` (`windows/scripts/Write-EnvFile.ps1:51`), invoked from `Setup-Wizard.ps1:998-1002` when the user clicks **Start Installation**	Yes — the backend container must (re)start to pick up a `.env` change, because settings are read once into a process-lifetime singleton (see below).
"Manage Providers" web UI (`/providers`) → `POST /api/v1/runtime/ai-providers/{backend}` or `POST /api/v1/runtime/ioc-providers/{provider_id}`	`provider_runtime_configs` table in Postgres, Fernet-encrypted	`backend/app/api/routes/runtime.py:38-48` (`configure_ai_provider`, calling `svc.upsert_ai_provider`) and `runtime.py:135-152` (`configure_ioc_provider`, calling `svc.upsert_ioc_provider`); frontend side is `configureAIProvider` / `configureIOCProvider` in `frontend/lib/api.ts:418-423` and `:482-487`, called from `frontend/app/providers/page.tsx:106-174`	No — the very next investigation or AI call after the save picks up the new value.

Both configuration-mutating endpoints are gated by `require_permission("provider:manage")`. Wizard-collected values are sanitized before being written to disk: `ConvertTo-SafeEnvValue` (`Write-EnvFile.ps1:15-49`, applied to every field at `:62-66`) strips carriage returns/newlines and rejects any value containing `#`, since an unescaped `#` starts a comment in `.env` syntax and would silently truncate the rest of the line.

A third, deliberately non-persisting action exists on both sides: `POST /api/v1/providers/{provider_id}/test` (`backend/app/api/routes/providers.py:20-36`) and `POST /api/v1/ai/test` (`backend/app/api/routes/ai_config.py:39-56`) each make one real, minimal outbound call using whatever credential is currently sitting in the request body — never a value already saved anywhere — and never write it to `.env` or to the database. These are the endpoints behind every "Test Connection" button in both the wizard and the web UI.

The Legacy Design: a Frozen Settings Singleton

Before the runtime layer existed, every provider's and every AI backend's configuration was `.env`-driven and fixed for the entire lifetime of the running backend process:

- `app/core/config.py:13` defines `class Settings(BaseSettings)` (pydantic-settings), populated once from `.env` and the process environment.
- `app/core/config.py:110-111` wraps construction in `@lru_cache def get_settings()` — the first call builds the one and only `Settings` instance for the process; every subsequent call anywhere in the codebase returns that same object, forever, until the process restarts.
- Each provider computed its own `configured` boolean **once, at module-import time**, directly from that frozen singleton — e.g. VirusTotal's connector reads `bool(get_settings().virustotal_api_key)` in its constructor. There was no code path that ever re-evaluated this flag after the process started.

The practical consequence: editing `.env` — whether by hand or via the wizard — had no effect on a backend process that was already running. This is the entire reason the runtime-config system described in the rest of this chapter exists, and it is also the exact root cause of a specific, documented behavioral gap covered in its own section below.

The Current Design: Encrypted, Database-Backed, Runtime-Mutable

Storage: `provider_runtime_configs`

`backend/app/models/runtime_config.py:30-55` defines `ProviderRuntimeConfig`, one row per `(kind, provider_id)` pair (`kind` is `ai` or `ioc`; unique constraint at `runtime_config.py:32`). The column that matters most for this chapter is `encrypted_credentials: Mapped[Optional[str]]` (`runtime_config.py:42`) — a `TEXT` column that only ever holds Fernet ciphertext, never plaintext. Alongside it: `enabled`, `is_active` (meaningful only for AI rows — exactly one AI row is meant to be active at a time, enforced in application code, not a database constraint), `model_id`, `extra_config` (JSONB, for non-secret extras such as a custom endpoint URL), `last_test_at` / `last_test_ok` / `last_test_message` (so the UI can show "last tested" status without re-testing), and `updated_by` (FK → `users.id`).

A second table, `config_audit_log` (`runtime_config.py:58-75`), records every configure/enable/disable/activate/test-result action: `timestamp`, `actor_user_id` (FK → `users.id`), a denormalized `actor_email` (so the log stays readable even if the account is later deleted), `action`, and a free-text `detail` field. The write path for `detail` only ever accepts descriptive text — it is not a location where a credential value can end up. `GET /api/v1/runtime/audit-log` (`runtime.py:185`, `require_permission("audit:read")`) exposes this table.

Encryption: `app/core/crypto.py`

Credentials are Fernet-encrypted (symmetric, `cryptography.fernet.Fernet`) before they are written anywhere:

- `encrypt_secret()` (`crypto.py:49-54`) returns Fernet ciphertext, or `""` for falsy input.
- `decrypt_secret()` (`crypto.py:57-67`) returns `""` on any `InvalidToken` rather than raising — a corrupted or key-mismatched ciphertext degrades to "no credential" instead of crashing a request.
- The Fernet key itself comes from `_fernet()` (`crypto.py:39-46`): an explicit `encryption_master_key` setting (`config.py:30`) if the operator has set one, otherwise an HKDF-derived key from the existing `jwt_secret_key` (`config.py:23`, derivation at `crypto.py:33-36`). The HKDF fallback means every pre-existing installation already has a working encryption key with no migration

step — but it also means that, absent an explicit `encryption_master_key`, compromising `jwt_secret_key` would expose the derived encryption key too. Operators wanting independent key separation must set `encryption_master_key` explicitly.

- `mask_secret()` (`crypto.py:70–77`) produces the `****…abcd`-style preview the UI displays; `_row_to_public_dict` (`runtime_config.py:167`) is the only place a stored credential is ever rendered back to a caller, and it always goes through this masking — no API response returns a decrypted credential.

`upsert_ai_provider` (`runtime_config.py:237`) and `upsert_ioc_provider` (`runtime_config.py:357`) both merge the incoming `credentials` dict onto whatever is already decrypted and stored (`_merge_credentials()`, `runtime_config.py:121–140`) before calling `encrypt_secret(json.dumps(merged))` and assigning `row.encrypted_credentials` — the encryption step happens on every write, with no unencrypted code path. The merge (rather than a blind overwrite) matters specifically because the settings UI's per-field inputs only ever populate a field the operator actually retypes that session — a plain overwrite with an empty or partial payload used to silently wipe whichever fields weren't retyped; see the regression tests in `app/tests/unit/test_runtime_config.py` and `app/tests/integration/test_runtime_config_persistence.py`.

Bridging old installs: `seed_from_env_if_empty()`

`runtime_config.py:443–493` copies the legacy `.env` values into the new table exactly once, the first time it ever runs on a given database: if `provider_runtime_configs` already has any row at all, it is a no-op (`:454–455`, so it never overwrites a value an administrator has already configured through the UI); otherwise, for each of the 11 AI backends (`_AI_ENV_SEED_MAP`, `:62–76`) and each of the 18 registered IOC providers (`_ENV_SEED_MAP`, `:49–60`, sourced from `app.providers.registry.get_all_providers()`), it reads the matching field off `get_settings()` and inserts a pre-encrypted row (`:458–472`, `:476–488`). It is invoked once per process start, from `app/main.py:50–58` (`@app.on_event("startup") async def _seed_runtime_config()`), wrapped in a try/except so a seeding failure never blocks boot (`main.py:59–60`). The function's own docstring states it is "idempotent and safe to call on every startup" (`runtime_config.py:448`) — this is what lets an existing `.env`-configured deployment upgrade to the new system without losing its working credentials or requiring a manual one-time migration script.

The Runtime API Surface

All ten endpoints below live in `backend/app/api/routes/runtime.py`, prefix `/api/v1/runtime`:

Endpoint	Permission	Purpose
`GET /ai-providers`	`provider:manage`	List every AI backend's persisted runtime row (masked credentials).
`POST /ai-providers/{backend}`	`provider:manage`	Persist credentials/model for one AI backend (`svc.upsert_ai_provider`).
`POST /ai-active`	`provider:manage`	Switch the platform-wide active AI backend immediately.
`GET /ai-active`	`lookup:read`	Read the currently active AI backend + model.
`POST /ai-providers/{backend}/record-test`	`provider:manage`	Record a prior `/ai/test` outcome against the saved row.
`GET /ioc-providers`	`provider:manage`	List every known IOC provider merged with its runtime row (or a synthesized default).
`POST /ioc-providers/{provider_id}`	`provider:manage`	Persist credentials/extra config for one IOC provider.
`POST /ioc-providers/{provider_id}/enabled`	`provider:manage`	Enable/disable a provider platform-wide, effective on the next investigation.
`POST /ioc-providers/{provider_id}/record-test`	`provider:manage`	Record a prior `/providers/{id}/test` outcome.
`GET /audit-log`	`audit:read`	Retrieve the `config_audit_log` history.

Live Retrieval at Investigation Time: Two Different Mechanisms

Reading a credential back at the moment it is needed is handled differently for IOC providers than for AI backends, and the difference is deliberate.

IOC providers: a per-investigation `ContextVar` snapshot

Every IOC provider connector (`app/providers/*.py`) is instantiated exactly once at import time and reused as a long-lived module-level singleton for the entire life of the process (`app/providers/registry.py`). That singleton is shared by every concurrent investigation the backend is handling at any given moment — which makes it unsafe to mutate directly.

Why a naive "mutate the singleton" approach would be a concurrency bug: if an administrator's `POST /ioc-providers/{id}` call (or the enable/disable toggle) simply wrote the new credential/enabled state onto the shared provider instance's own attributes, then two investigations in flight at the same time — one that started just before the change and is still mid-flight, another that starts just after — would both read from the *same* mutable object. There is no guarantee which investigation's provider task executes its credential read before or after the write lands, so the in-flight investigation could non-deterministically pick up the *new* credential mid-run (attributing its results to the wrong configuration state), or the administrator's own just-saved change could be transiently overwritten if two admin requests raced each other. Either way, "investigation A used configuration X" would stop being a reliable statement.

The fix is `app/core/runtime_context.py`, which never mutates the provider objects at all:

1. `_provider_overrides` (`runtime_context.py:19–21`) is a `contextvars.ContextVar[Optional[dict]]`, not an attribute on any provider.
2. At the start of every single investigation, before any per-provider task is spawned, `run_all_providers()` (`app/providers/orchestrator.py:112–113`) does exactly one fresh Postgres read — `snapshot = await get_ioc_provider_snapshot()` (`runtime_config.py:336–354`) — and calls `set_provider_overrides(snapshot)` to bind that snapshot into the current context.
3. Immediately after, `orchestrator.py:120` calls `asyncio.create_task()` once per eligible provider. Because `asyncio.create_task()` copies the *current* `contextvars.Context` into every task it spawns, every concurrent per-provider task belonging to that one investigation inherits the identical, frozen snapshot — regardless of what any administrator does to the underlying table while those tasks are still running. This is the concurrency-safety property the module's own docstring states outright (`runtime_context.py:1–14`).
4. `BaseProvider.run()` (`app/providers/base.py:104–111`) reads the override for its own `provider_id` from that context (`get_provider_override(self.provider_id)`); if the effective `configured` is false, it short-circuits (`base.py:135–146`) with `ProviderStatus.NOT_CONFIGURED` and the message `f"{self.provider_name} is not configured (missing API key/credentials)."` (`base.py:143`) before making any outbound call.
5. Inside `fetch()`, each connector reads its actual credential value through `get_credential(provider_id, field, fallback)` (`runtime_context.py:38–47`) — e.g. `app/providers/virustotal.py:44`: `api_key = get_credential("virustotal", "api_key", settings.virustotal_api_key)`. It returns the per-investigation override if one was set, otherwise the legacy `.env`-derived value, so an unconfigured runtime row degrades gracefully to whatever `Settings` still holds.

One immutable snapshot per investigation, read-only for the duration of that investigation, is the entire mechanism — no locking is needed because nothing shared is ever written to after the snapshot is taken.

AI backends: a fresh client per call, not a `ContextVar`

The AI path does not need the snapshot pattern at all, because it takes a simpler route: it never keeps a long-lived, shared, mutable client instance in the request path in the first place. `_get_ai_client()` (`app/ai/service.py:102–145`) resolves the backend in three tiers — an explicit `backend_override` argument (used only by the AI-comparison "re-analyze with a different backend" feature, `service.py:125–126`) → the active runtime config, read fresh from Postgres on **every single call** via `get_active_ai_config()` (`service.py:128` , backed by `runtime_config.py:200–215` , which is not cached) → the legacy `Settings` singleton as a final fallback if no runtime row exists yet (`service.py:134–138`). `_build_client()` (`service.py:56–99`) then constructs a **brand-new client instance** for that one call — e.g. a fresh `AnthropicClient(api_key=..., model_id=...)` (`app/ai/anthropic_client.py:25–39`) — deliberately bypassing the module-level frozen singleton client (`get_anthropic_client()` , `anthropic_client.py:93–97`) that only reflects `Settings` . Because every call re-reads the database and rebuilds its own client from that read, switching the active AI backend via `POST /api/v1/runtime/ai-active` takes effect on the very next AI call made by *any* investigation, with no restart and no shared mutable state to protect. If the resolved client's `is_configured` is false, the call fails loudly: `RuntimeError(f"AI backend '{backend}' is not configured (missing API key/URL/model) -- configure it from the AI Providers panel or check .env")` (`service.py:140–144`).

A Real, Documented Historical Bug: "Test Connection: SUCCESS" but the Investigation Still Failed

The codebase and its accompanying repo-root reports independently corroborate one specific, real architectural bug that predates the runtime-config system, distinct from the mechanisms above.

Root cause. `app/providers/connection_test.py` 's own module docstring (`:1–16`) states it plainly: connection-test handlers are "deliberately separate from the `BaseProvider.fetch()` code path," because `fetch()` reads the frozen `get_settings()` singleton (fixed for the process's lifetime), so there was no safe way to test a key the user had just typed into a form without either restarting the process or mutating shared global state under concurrent requests — so the test handlers make one real HTTP call with the *candidate* key and never touch `get_settings()` at all. Meanwhile, before the runtime-config system existed, each provider's `configured` flag was computed exactly once, at module-import time, from that same frozen singleton (e.g. `virustotal.py:24–26` : `self.configured = bool(get_settings().virustotal_api_key)`). The result: `POST /api/v1/providers/{id}/test` would correctly report success the instant a valid key was typed in — because it bypassed `get_settings()` — while a real investigation, routed through `BaseProvider.run()` , still saw the `configured` value frozen at process start. If the wizard wrote a new key to `.env` while the backend process was already running (the normal case, since Test Connection is meant to happen *before* a restart), the live investigation kept reporting `ProviderStatus.NOT_CONFIGURED` / `"is not configured (missing API key/credentials)"` immediately after a successful Test Connection.

This is independently confirmed in three separate, git-independent artifacts: `docs/PROVIDERS.md:375–378` ("Setting an API key in `.env` after the backend process has started will not retroactively flip `configured` for that provider — a process restart is required"), `docs/TROUBLESHOOTING.md:96–134` (same root cause under "A provider shows `not_configured` "), and `RUNTIME_PROVIDER_IMPLEMENTATION_REPORT.md:11–12` at the repo root ("Every AI backend, every provider's credential, and every 'configured' flag was read from this frozen snapshot… This was confirmed as the sole root cause [of the original limitation], not scattered logic, before any code was written.").

The fix is exactly the two mechanisms documented above in this chapter: the `ContextVar` -based per-investigation snapshot (`runtime_context.py` , fed by a fresh DB read in `orchestrator.py:112`) replaced the frozen `self.configured` as the value `base.py:109` actually checks, and `ai/service.py` 's `_get_ai_client()` replaced the frozen AI-client singleton with a fresh DB read plus a freshly-built client on every call. A credential saved through the Manage Providers UI today is therefore picked up by the very next investigation with no restart — closing the gap described above.

Two things worth flagging so this is not overstated. First, a *different* bug, with the opposite symptom (Test Connection *failing* despite a genuinely valid key), also exists and is separately self-documented in this codebase: every Test Connection button's request in the wizard required a session token (`$State.AccessToken`) that was previously only ever set inside the Summary page's "Start Installation" handler (`Setup-Wizard.ps1:211–236` , marked with an in-code "ROOT CAUSE" comment), so on a reconfigure run with no fresh login every Test Connection click failed with a "create the administrator account first" message even though the account and platform were already running — fixed by `Get-OrCreateWizardSession` (`Setup-Wizard.ps1:237–253`) and documented in `API_CONFIGURATION_FIX_REPORT.md:15–30` at the repo root. This is a distinct bug from the frozen-settings issue above and should not be conflated with it. Second, `test-provider-pipeline.sh` at the repo root contains a comment describing itself as reproducing "the reported 'API key test' bug using the REAL key already configured in the running container's own environment"; the script itself only re-runs `POST /providers/{id}/test` with various keys, and no further write-up tying that specific script to a named root cause was found beyond the frozen-settings-singleton issue documented above — it is noted here as existing evidence of *some* reported issue along these lines, not asserted to be definitively the same bug.

For completeness: the exact phrase "API key not provided" does not appear anywhere in this codebase's source or documentation (confirmed by a case-insensitive search of `backend/app` , `docs/` , `documentation/` , and `windows/` , excluding vendored dependencies). The real, verified error strings are `f"{self.provider_name} is not configured (missing API key/credentials)."` (`base.py:143`) and `f"AI backend '{backend}' is not configured (missing API key/URL/model) -- ..."` (`service.py:141–144`).

Before / After Summary

Aspect	Legacy (`.env` only)	Current (runtime-config layer)
Storage	Plaintext in `.env` on disk	Fernet-encrypted ciphertext in `provider_runtime_configs.encrypted_credentials`
Read by application code	`get_settings()` , an `@lru_cache` singleton fixed for the process's lifetime	Fresh Postgres read per investigation (IOC providers) or per AI call (AI backends)
Effect of a change	Requires editing `.env` and restarting the backend container	Effective on the very next investigation/AI call, no restart
Concurrency safety mechanism	None needed/possible — value never changed at runtime	`ContextVar` snapshot per investigation (IOC) / fresh client construction per call (AI)
Audit trail	None	`config_audit_log` table, one row per configure/enable/disable/activate/test-result action
Still in use today?	Yes — `Settings` remains the fallback when no runtime row exists, and is what `seed_from_env_if_empty()` reads from	Yes — the primary path once any row exists

Diagram

Figure 5 — Diagram: Diagram

Diagram: Credential lifecycle from wizard/UI entry through encrypted storage to per-investigation retrieval. Each investigation binds one immutable snapshot to its own `contextvars.Context` before spawning provider tasks, so two investigations racing across a credential change each see a single consistent configuration for their whole run -- never a mix of old and new mid-flight.

Testing and the Build Pipeline

This chapter documents three independent, non-overlapping pieces of tooling a maintainer needs to know: the backend's **pytest** suite (what exists, how it mocks the outside world, and the exact gotchas you will hit running it locally on this machine), the **Docker Compose / Windows Inno Setup** pipeline that turns the source tree into a running stack or an installable `.exe`, and the **documentation build pipeline** under `documentation/build/` that renders the very manual this chapter belongs to. There is no CI system anywhere in the repository -- no `.github/workflows/`, no `Makefile`, no other automation entry point was found -- so every command below is something a developer runs by hand.

1. Backend Test Suite Layout

Tests live under `backend/app/tests/`, split into `unit/` and `integration/`, each a real Python package (`__init__.py` present). **There is no `pytest.ini`, `pyproject.toml`, `setup.cfg`, `tox.ini`, or `conftest.py` anywhere in the repository** -- confirmed by a direct search of the tree. Pytest therefore runs with bare default discovery: no custom markers, no `asyncio_mode=auto` (every async test is decorated explicitly with `@pytest.mark.asyncio`, pytest-asyncio's strict-mode requirement), and no coverage gate, even though `pytest-cov` is a pinned dependency (`requirements.txt:29`) -- it is never invoked with `--cov` anywhere in the source, so coverage measurement is available but unused.

Layer	Location	Files	Lines (incl. blanks)
Unit	`backend/app/tests/unit/`	16	~1,800
Integration	`backend/app/tests/integration/`	3	~1,015

A direct count of `def test_...` / `async def test_...` function definitions across those 19 files puts the suite at **144 automated test functions** -- independently consistent with the "144 tests total" figure a later engineering pass reports after adding one of those files (`test_ai_connection_test.py`; see the Testing and Quality Assurance appendix for that pass's own account).

1.1 Unit tests (`backend/app/tests/unit/`)

No database, Redis, Docker, or network access is required for any file in this directory.

File	What it exercises
`test_abusech.py`	`app.providers.abusech.map_query_status` -- the shared abuse.ch `query_status` mapping used by URLhaus/ThreatFox/MalwareBazaar
`test_ai_connection_test.py`	`app.ai.connection_test.test_ai_connection` for all 11 AI backends against **respx**-mocked HTTP: success, invalid-key/401, rate-limit/429, model-not-found/404, timeout, network error, plus backend-specific cases (Kimi's thinking-mode conflict, DeepSeek's 402, xAI's flat error body) (53 test functions)
`test_ai_schemas.py`	`app.ai.schemas.MitreMapping`'s enum/regex-free `technique_id` field and `FinalAssessment`'s grounding validator
`test_ai_service.py`	`app.ai.service.generate_final_assessment`'s no-real-evidence short-circuit -- the fix for a fabricated `highly_malicious` verdict against the EICAR hash with zero provider evidence
`test_analysis_service.py`	`app.ai.analysis_service._strip_invalid_evidence_ids`, the guard that drops any AI-cited `evidence_id` not present in the real evidence set
`test_connection_test.py`	`app.providers.connection_test.test_provider_connection` for the keyed real providers, against **respx**-mocked HTTP
`test_correlation_engine.py`	`app.correlation.engine.correlate` -- edge dedup/merge and provider-agreement bucketing from fake `ProviderResult` envelopes, no I/O
`test_crawler_collector.py`	`InternetIntelligenceCollector.fetch` -- distinguishing a rate-limited OSINT source from a genuinely empty result
`test_crawler_rate_limit.py`	`app.crawler.sources.rate_limit.AsyncMinIntervalLimiter` timing behavior
`test_crypto.py`	`app.core.crypto.encrypt_secret` / `decrypt_secret` / `mask_secret` (Fernet round-trip, invalid-token handling, masking)
`test_evidence_builder.py`	`app.evidence.builder` -- the deterministic, non-AI evidence-ledger extraction every later AI explanation must cite
`test_ioc_detector.py`	`app.ioc.detector.detect_ioc_type` across many IOC string patterns
`test_pivot.py`	`app.evidence.pivot.rank_pivots` -- pure ranking logic behind `GET /lookup/{id}/pivots`
`test_provider_base.py`	`app.providers.base.BaseProvider` contract: unsupported IOC types, missing credentials, error normalization
`test_runtime_context.py`	`app.core.runtime_context`'s per-investigation `ContextVar` snapshot/override mechanics
`test_whois_rdap.py`	The WHOIS half of `app.providers.whois_rdap.WhoisRdapProvider`, including socket-failure vs. genuine-no-record handling

1.2 Integration tests (`backend/app/tests/integration/`)

File	Covers	Infra required
`test_api_health.py`	`GET /health`, `GET /docs`, and that the full OpenAPI schema (every DB-backed router included) serializes at app-construction time	None
`test_lookup_flow.py`	`app.providers.orchestrator.run_all_providers` / `run_all_providers_collected` fan-out, registered against **fake `BaseProvider` subclasses** via the orchestrator's `providers=` override parameter -- the real provider registry is never imported	Redis (`localhost:6379`)
`test_lookup_stream_persistence.py`	Regression coverage for a bug where every lookup stayed `status=RUNNING` forever because the route's DB session was torn down before the lazy SSE generator ran	Postgres (`localhost:5433`) + Redis (`localhost:6379`)

`test_lookup_flow.py` and `test_lookup_stream_persistence.py` each perform a live TCP reachability probe at import time and use `pytest.mark.skipif` to **skip the whole module** (not fail the run) when the required service isn't reachable -- so running the suite with no Docker services up still passes, it just silently exercises fewer files.

2. Mocking and Fixture Conventions

Three patterns recur across the suite and are worth knowing before adding a new test:

- **respx for HTTP.** `test_ai_connection_test.py` and `test_connection_test.py` mock outbound calls with `@respx.mock` decorators against the real vendor URLs (e.g. `respx.post("https://api.groq.com/openai/v1/chat/completions").mock(return_value=httpx.Response(429))`) -- every credential value in these files is a placeholder string, never a real key. `test_lookup_flow.py` uses the same library for any HTTP its fake providers happen to trigger.
- **Monkeypatched AI client accessor, not the AI client itself.** Rather than mocking an HTTP call, `test_ai_service.py` replaces the *resolver function* directly: `monkeypatch.setattr("app.ai.service._get_ai_client", _fail_if_called)`, where `_fail_if_called` raises `AssertionError` if it is ever invoked -- proving the no-evidence short-circuit returns `verdict=unknown` without reaching any AI backend at all. `test_lookup_stream_persistence.py` uses the same seam the opposite way, substituting a `_StubAIClient` so the integration test never depends on a real Ollama/Groq/etc. connection: `monkeypatch.setattr(ai_service, "_get_ai_client", _stub_get_ai_client)`.
- **Fake providers via a constructor parameter, not monkeypatching.** `test_lookup_flow.py` and `test_lookup_stream_persistence.py` both pass `providers=[...]` (a list of hand-written `BaseProvider` subclasses) straight into the orchestrator functions, so the real 18-provider registry is never imported by these tests -- there is no risk of a real outbound call to VirusTotal, AbuseIPDB, etc. leaking into the suite.

There is no shared `conftest.py`. Both infra-dependent integration files independently reimplement near-identical Postgres/Redis-override and connection-pool-disposal fixtures rather than sharing one -- a real, if minor, duplication a maintainer adding a fourth integration file should be aware of before copy-pasting a fifth copy of the same fixture.

There is no dedicated test for auth's HTTP endpoints. `POST /api/v1/auth/register` and `POST /api/v1/auth/login` are exercised only indirectly, via `User` rows and tokens constructed directly inside `test_lookup_stream_persistence.py`'s fixtures.

3. Running the Backend Tests

For integration coverage, start the two infra services first (unit tests need neither):

```
docker compose up -d postgres redis
```

Then, from `backend/`:

```
cd backend
pytest                      # everything default-discovery finds
pytest app/tests/unit       # unit only
pytest app/tests/integration # integration only
```

3.1 Two drifted local virtualenvs

Two backend virtualenvs exist on disk in this repository, `backend/.venv` and `backend/.venv_test`; neither is committed, and which one is the "intended" one for running tests is not documented anywhere. Directly comparing both against the versions actually pinned in `requirements.txt` (verified with `pip show` in each, at time of writing):

Package	Pinned (`requirements.txt`)	`backend/.venv`	`backend/.venv_test`
pytest	8.3.3	9.1.1	8.3.3
pytest-asyncio	0.24.0	1.4.0	0.24.0
bcrypt	4.0.1	4.0.1	5.0.0
asyncpg	0.29.0	0.31.0	0.30.0
cryptography	43.0.1	**not installed**	50.0.0
python-whois	0.9.6	0.9.6	0.9.4

Running the full suite (`pytest --continue-on-collection-errors`, bare, from `backend/`) against each venv, with the Postgres/Redis containers from `docker-compose.yml` up, produced:

- `.venv`: **150 passed, 9 errors**
- `.venv_test`: **153 passed, 9 errors**

Every single one of those 9 errors decomposes into just two root causes, not nine unrelated bugs:

1. **Two are a pytest-discovery false positive, present in both venvs, unrelated to any dependency drift.** `app/ai/connection_test.py` and `app/providers/connection_test.py` are *production* modules (live candidate-credential checkers used by `routes/ai_config.py` and `routes/providers.py`), not test files -- but their filenames end in `_test.py` (pytest's default `*_test.py` discovery pattern) and each defines a top-level function literally named `test_ai_connection(backend, credentials, model=None)` / `test_provider_connection(provider_id, credentials)`. With no `testpaths` restriction anywhere in the repo, bare `pytest` tries to collect and run these as real tests, and both fail identically at setup with `fixture 'backend' not found` (or `'provider_id'`) -- not a defect in the application, just a naming collision with pytest's default discovery. Scoping the run to `pytest app/tests` (as shown above) avoids this entirely.
2. **The remaining 7 trace to exactly one drifted or missing package per venv**, because the affected module sits on the import path of nearly the whole app. In `.venv`, `cryptography` is not installed at all, and `app/core/crypto.py` imports it unconditionally -- this breaks collection of `test_crypto.py` directly, and also `test_api_health.py` / `test_lookup_flow.py`, since both import the FastAPI app object, which transitively imports the runtime-config module, which imports `crypto.py`. In `.venv_test`, `python-whois==0.9.4` is installed instead of the pinned `0.9.6`; `0.9.4` has no `whois.exceptions` submodule, and `app/providers/whois_rdap.py:23`'s `from whois.exceptions import PywhoisError` fails -- breaking collection of `test_whois_rdap.py` directly, `test_api_health.py` / `test_lookup_flow.py` the same transitive way, and causing 4 of `test_lookup_stream_persistence.py`'s 5 tests to fail specifically at the `monkeypatch.setattr("app.api.routes.lookup...", ...)` setup step, since that import chain also runs through the provider registry.

This is a distinct, additional pair of environment gotchas beyond the bcrypt/passlib version issue and the asyncio event-loop-pool issue already written up in `docs/TESTING.md` (both of which remain accurate: `bcrypt>=4.1`'s missing `__about__` attribute breaks passlib's `CryptContext.hash()` reproducibly in `.venv_test`'s bcrypt 5.0.0, and both infra-touching integration files carry their own `autouse` pool-disposal fixtures specifically to avoid "Event loop is closed" errors on the second test in a module). The practical takeaway for a maintainer: **before trusting any local pytest run's pass**

count on this project, reconcile whichever venv you're using against `requirements.txt` first -- the drift is real, silent, and (as shown above) cascades far wider than the one file that actually exercises the missing/wrong dependency.

4. Frontend Tests

`frontend/package.json` declares `"test": "vitest run"` and lists `vitest@2.1.1` as a devDependency, but a repository-wide search for `*.test.*` / `*.spec.*` under `frontend/` finds zero files, and no `vitest.config.*` exists either. Running `npm test` in `frontend/` executes Vitest against an empty suite. This is configured-but-unused tooling, not a working test suite -- see the Testing and Quality Assurance appendix for the fuller picture, including the separate manual QA pass that exercises the product end-to-end where automated frontend tests do not.

5. Docker Compose Build Pipeline

`backend/Dockerfile` and `frontend/Dockerfile` are both single-stage (`python:3.12-slim` with `gcc` / `libpq-dev` / `curl` , and `node:20-alpine` , respectively) -- there is no multi-stage build anywhere in the repository; each image simply installs dependencies from `requirements.txt` / `package.json` and copies the full source tree in.

`docker-compose.yml` defines the eight containers described in the Technical Architecture Overview (`postgres` , `redis` , `neo4j` , `opensearch` , `backend` , `celery_worker` , `celery_beat` , `frontend`); in its current form every datastore port is bound to `127.0.0.1` only, and each of the four datastore services carries an inline comment explaining that a `0.0.0.0` -published equivalent was confirmed reachable, unauthenticated, from another device on the same LAN before this fix. For local development the `backend` / `frontend` services bind-mount the source tree and run `uvicorn ... --reload` / `npm run dev` .

`docker-compose.prod.yml` is a Compose **override** file, layered on top with `-f` , used for anything other than a live-reload dev checkout (this is the file the Windows installer drives):

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

It resets every bind-mounted `volumes:` list to empty (`!reset []` , so each container runs entirely from what was baked into its image), and swaps the dev commands for: `alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000` for `backend` , and, for `frontend` , a multi-step shell command that runs `npm run build` , manually copies `.next/static/` into `.next/standalone/` (Next.js's `output: "standalone"` build intentionally excludes it), and finally launches `node .next/standalone/server.js` -- a workaround documented inline for a confirmed-live failure where `next start` refuses to run against a standalone build at all.

There is no k8s CI/build integration confirmed in the source; `k8s/` exists as a parallel Kustomize-based deployment path (StatefulSets/Deployments/Ingress) but is not otherwise inspected here.

6. Windows Installer Build

`windows/installer.iss` (359 lines, Inno Setup 6 script) is compiled with:

```
ISCC.exe installer.iss
```

producing `release\HORIZON-GRID-Setup-{version}.exe` (both the `OutputBaseFilename` and the version string come from the `#define MyAppVersion` at the top of the script, currently `"0.2.5"`). The `[Files]` section stages `backend/` , `frontend/` , both compose files, `k8s/` , `docs/` , and `README.md` into the package, with explicit excludes on each side: `__pycache__,*.pyc,.pytest_cache,celerybeat-schedule,nul` for `backend/` (the trailing `nul` exclude exists because a stray file literally named `nul` -- a reserved Windows device name, left behind by some earlier `> nul` redirect -- otherwise aborts Inno Setup's compressor, which can't read its file time) and `node_modules,.next,*.tsbuildinfo` for `frontend/` . The installer requires 64-bit-compatible Windows (`ArchitecturesAllowed=x64compatible`) and admin privileges (`PrivilegesRequired=admin`); the runtime behavior of the resulting installer (prerequisite checks, the WinForms setup wizard's page flow, Program Files/ProgramData layout, and Start Menu shortcuts) is covered in full in the Windows Deployment Architecture chapter and is not repeated here -- this section is scoped to how the installer artifact itself gets built.

7. Documentation Build Pipeline (`documentation/build/`)

The documentation package you are reading is itself produced by a small Node.js toolchain, explicitly marked in its own `package.json` as `"Local build tooling for rendering the final PDF/DOCX documentation deliverables. Not part of the product."` Its dependencies: `puppeteer-core` (drives a local, already-installed Chrome or Edge to render HTML to PDF/PNG -- no bundled Chromium download), `marked` (Markdown to HTML), `mermaid` (diagram rendering), `html-to-docx` + `docx` + `@xmldom/xmldom` + `jszip` (DOCX generation, including hand-editing the generated `.docx` 's internal `word/document.xml` to inject a real, updatable Word Table-of-Contents field), `image-size` (figure-sizing decisions), and `pdf-parse` (reading rendered PDF text back out, described below).

Two generations of build script coexist:

- The original four (`assemble.js` , `build-html.js` , `render-pdf.js` , `build-docx.js`) read from a pre-baked cache, `sections.json` , which was assembled once from all `tech-*.md` and `user-*.md` files, and produce the combined `FINAL_PRODUCT_DOCUMENTATION.pdf` / `.docx` .
- The newer, generic `build-doc-generic.js` reads `DOCUMENTATION_SOURCE/*.md` **fresh on every run** (no cache) and is driven by one of three small config files -- `config-user-manual.json` , `config-backend.json` , `config-source-code.json` -- each listing an ordered `files` array. This chapter, `dev-06-testing-and-build.md` , is entry 6 of 8 in `config-source-code.json` 's list, which builds `IOC_INTELLIGENCE_PLATFORM_SOURCE_CODE_DOCUMENTATION.pdf` / `.docx` . It is invoked per deliverable:

```
cd documentation/build
npm install
node build-doc-generic.js config-source-code.json
```

Within a single run, `buildOne()` renders the PDF in **two passes**: pass 1 renders with a placeholder Table of Contents (each section's opening `<div>` carries an invisible `SECTIONMARK_N` marker span) to a throwaway PDF, which `pdf-parse` then re-reads page-by-page to resolve each marker's real printed page number; pass 2 re-renders the full document with those real page numbers filled into the Table of Contents. The FIGURE-placeholder convention used throughout the source chapters -- a bracketed tag naming an image filename and a caption, separated by a pipe -- is resolved by `resolveFigurePath()` : any filename matching `*-diagram-N.png` is loaded from `documentation/ARCHITECTURE_DIAGRAMS/` , everything else from `documentation/SCREENSHOTS/` ; a missing file does not abort the build -- it renders a red inline missing-figure marker and logs an error to the console instead.

Diagrams are authored as fenced ```` ```mermaid ```` code blocks directly inside a `DOCUMENTATION_SOURCE/*.md` chapter, then converted with a separate script:

```
node render-diagrams.js
```

This scans every file matching `(tech|user|dev|backend)-\d+.*\.md` for ```` ```mermaid ```` blocks, renders each one via headless Chrome + `mermaid.js` to a standalone PNG named `{chapter-basename}-diagram-{n}.png` under `ARCHITECTURE_DIAGRAMS/`, and **rewrites the source chapter file in place**, replacing the raw Mermaid block with a resolved FIGURE placeholder captioned from the nearest preceding heading -- meaning a chapter's committed Markdown, once processed, never contains the original diagram source, only the rendered-image reference.

Summary

The backend carries a real, if partial, automated safety net -- 144 pytest test functions across 16 unit and 3 integration files, using respx for HTTP mocking, monkeypatched AI-client accessors and hand-written fake providers to keep the suite offline and deterministic, and live-reachability `skipif` guards so infra-dependent tests degrade to "skipped" rather than "failed" when Postgres/Redis aren't up. That safety net sits on top of a genuinely inconsistent local environment story, though: two undocumented, uncommitted virtualenvs, each drifted from `requirements.txt` in a different way, plus a pytest-discovery quirk that mistakes two production modules for test files -- all independently reproduced above. The frontend has test tooling configured and zero tests to run. Shipping the product itself runs through two independent, uncoordinated pipelines with no CI gluing them together: `docker compose ... up -d --build` (optionally with the `docker-compose.prod.yml` overlay) for the running stack, and `ISCC.exe installer.iss` for the Windows-installable artifact that automates driving that same Compose command. This very documentation set is produced by a third, separate pipeline again -- a small, explicitly-not-part-of-the-product Node toolchain under `documentation/build/` that renders each audience-specific chapter list into a two-pass, page-numbered PDF and a TOC-enabled DOCX.

Extension Points: Adding Providers, AI Backends, Endpoints, and Migrations

This chapter is a practical, step-by-step reference for the four kinds of change a maintainer is most likely to make to this codebase: adding a new IOC intelligence provider, adding a new AI backend, adding a new API endpoint, and adding a database migration. Each walkthrough cites the exact files, classes, and functions involved and points at an existing piece of code to use as a template. It assumes familiarity with the Provider Architecture, AI Architecture, and Database Architecture chapters elsewhere in this document, and focuses on *where to make the change*, not on re-explaining what each subsystem does.

1. Adding a New IOC Provider

Every provider is a subclass of `BaseProvider` (`backend/app/providers/base.py`), instantiated once as a module-level singleton and listed in `backend/app/providers/registry.py`. The orchestrator, correlation engine, and API layer never branch on which concrete provider answered — they only call the shared interface — so a fully working new connector is, by design, a self-contained, two-file change: the new connector module, plus one import and one list entry in `registry.py`.

Choose a template. For a keyless connector with a single simple HTTP GET, use `backend/app/providers/nvd.py` (`NVDProvider`): `requires_key = False`, `self.configured = True` set unconditionally in `__init__`, and a `fetch()` that is one `httpx.AsyncClient.get()` call, a 404-to-`NO_DATA` branch, and a `_map()` helper building the normalized `data` dict. For a provider requiring an API key sent as a header, use `backend/app/providers/virustotal.py` or `abuseipdb.py` — both read their key via `get_credential(provider_id, field_name, settings.<field>_api_key)` (`app/core/runtime_context.py`) rather than calling `get_settings()` directly, which is what makes the key hot-swappable at runtime. For a DNS-only connector with no HTTP call at all, use `backend/app/providers/stubs/spamhaus.py`.

Implement the class. At minimum, declare the class-level attributes `BaseProvider` expects and implement one async method:

Attribute / method	Type	Example (`nvd.py`)
`provider_id`	`str`, unique in the registry	`"nvd"`
`provider_name`	`str`, human-readable	`"NIST NVD"`
`category`	`ProviderCategory` enum (`base.py`)	`ProviderCategory.VULNERABILITY`
`supported_types`	`set[IOCType]` (`app/ioc/types.py`)	`{IOCType.CVE}`
`requires_key`	`bool`	`False`
`configured`	`bool`	`True`
`base_url`	`str`	`"https://services.nvd.nist.gov/rest/json/cves/2.0"`
`async fetch(self, ioc_value, ioc_type, client) -> ProviderResult`	method	—

`fetch()` must return a `ProviderResult` with `status` one of `OK`, `NO_DATA`, `ERROR`, `TIMEOUT`, `RATE_LIMITED`, or `UNSUPPORTED_IOC` — never raise for an expected "nothing found" case; return `NO_DATA` instead. Any `httpx.HTTPStatusError` that escapes `fetch()` is caught one layer up in `BaseProvider.run()` and mapped to `RATE_LIMITED` for HTTP 429/403/509 or `ERROR` otherwise, so a plain `response.raise_for_status()` needs no local try/except. Do **not** implement retry/backoff inside `fetch()` — that is applied uniformly in `backend/app/providers/orchestrator.py` (`asyncio.wait_for` + `tenacity`).

If the provider needs a credential, read it with `get_credential()`, not `get_settings()`:

```
from app.core.runtime_context import get_credential
api_key = get_credential("my_new_provider", "api_key", settings.my_new_provider_api_key)
```

This is the pattern every keyed provider uses, and is what allows the credential to be swapped in per-investigation from the runtime-config database (via the `ContextVar` snapshot set in `orchestrator.py`) rather than being frozen at process start. The fallback field (`settings.my_new_provider_api_key`) must exist on `Settings` in `backend/app/core/config.py`, following the naming convention of `virustotal_api_key` / `nvd_api_key`; it is only used before the runtime-config table has been seeded, or if no runtime override exists yet for that provider.

Register the provider in `backend/app/providers/registry.py`:

```
from app.providers.my_new_provider import my_new_provider

_ALL_PROVIDERS: list[BaseProvider] = [virustotal_provider, ..., my_new_provider]
```

That single list is the only place the orchestrator, `GET /api/v1/providers/health`, and `GET /api/v1/runtime/ioc-providers` look.

Register credential fields (if keyed). Add an entry to `IOC_PROVIDER_CREDENTIAL_FIELDS` in `backend/app/core/runtime_config.py` — `{"my_new_provider": ["api_key"]}` — which drives the dynamic field list on the Manage Providers UI's IOC Providers tab (`GET /api/v1/runtime/ioc-providers` appends `credential_fields = svc.IOC_PROVIDER_CREDENTIAL_FIELDS.get(provider_id, [])` to each row). Omitting it still lets the provider be enabled/disabled from the UI, but renders no credential input. For backward compatibility with an existing `.env` value, also add a matching entry to `_ENV_SEED_MAP` in the same file.

Add a live connection test (optional but expected for keyed providers). `backend/app/providers/connection_test.py` holds per-provider test handlers, dispatched from `POST /api/v1/providers/{provider_id}/test` — this powers the wizard's and Manage Providers UI's "Test" button, making one real outbound call with the *candidate* credentials from the request body, never the saved ones. A provider with no handler simply reports it requires no credential, as happens today for `crtsh`, `cisa_kev`, `mitre_attack`, `whois_rdap`, `spamhaus`, and `internet_intelligence`.

Nothing else needs to change: the orchestrator, correlation engine, evidence builder, and every API route operate purely off the `BaseProvider` interface and the `_ALL_PROVIDERS` list — exactly the guarantee `registry.py`'s module docstring states: "adding a new provider is a two-line change (import + append) and never touches the orchestrator, API routes, or correlation engine."

---

2. Adding a New AI Backend

All eleven existing AI backends (Ollama, Anthropic, Bedrock, Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, OpenRouter) expose an identical async contract, defined structurally as the `_AIClient` `Protocol` in `backend/app/ai/service.py`:

```
class _AIClient(Protocol):
    is_configured: bool
    async def call_claude_json(
        self, system_prompt: str, user_prompt: str, json_schema: dict,
        tool_name: str = ..., max_tokens: int | None = ...,
    ) -> dict: ...
```

Nothing in `ai/service.py`, `ai/analysis_service.py`, or `ai/hunting_service.py` branches on which backend is active beyond resolving *which* client to construct.

Implement the client class. Create `backend/app/ai/my_backend_client.py` following `anthropic_client.py` (simplest real HTTP example) or `groq_client.py` (if the target API is OpenAI-tool-calling-compatible). It needs: a constructor `MyBackendClient(api_key: Optional[str] = None, model_id: Optional[str] = None, max_tokens: Optional[int] = None)` with every argument optional (falling back to `get_settings()` when omitted) — this dual mode is what lets `_build_client()` construct either a fresh, explicitly-credentialed instance or the legacy settings-derived singleton from the same class; an `is_configured` property; a `call_claude_json()` that forces structured JSON matching `json_schema` via whatever mechanism the backend supports — a forced tool call (Anthropic, Bedrock, Groq), a schema-constrained JSON response format (Gemini), or grammar-constrained decoding (Ollama's `format` field), flattening `$ref` / `$defs` first with `inline_refs()` (`app/ai/schema_utils.py`) if the schema dialect needs it; and a module-level singleton getter `get_my_backend_client()` mirroring `get_anthropic_client()` / `get_groq_client()`, returned by `_build_client()` when called with `credentials=None`. On any failure, raise `RuntimeError` — never return a partial or `None` result, since every caller treats `RuntimeError` as the uniform "this backend failed" signal.

Wire it into `_build_client()` in `backend/app/ai/service.py`:

```
if backend == "my_backend":
    from app.ai.my_backend_client import MyBackendClient, get_my_backend_client
    if credentials is None:
        return get_my_backend_client()
    return MyBackendClient(api_key=credentials.get("api_key"), model_id=model_id)
```

This is called from `_get_ai_client()`, the single 3-tier backend-resolution point used everywhere: explicit `backend_override` → the active runtime-configured backend (fresh DB read via `runtime_config.get_active_ai_config()`) → the legacy frozen `settings.ai_backend`. A fresh client instance is deliberately constructed on every call (never a cached singleton) so switching backends takes effect on the very next AI call, with no restart.

Register the backend name and its env-seed mapping. Add `"my_backend"` to `AI_BACKENDS` in `backend/app/core/runtime_config.py` — `POST /api/v1/runtime/ai-active` validates against this list, so without this entry the backend can never become active and the route returns `400 f"Unknown AI backend {backend!r}"`. Then add an entry to `_AI_ENV_SEED_MAP` in the same file so an existing `.env` value is picked up once, on first startup, by `seed_from_env_if_empty()` (idempotent — only runs when `provider_runtime_configs` has zero rows, from `main.py`'s startup hook; it will not backfill a row added after the table already has data — configure such a backend once via the Manage Providers UI instead):

```
AI_BACKENDS = ["ollama", "anthropic", "bedrock", "gemini", "groq", "my_backend"]
_AI_ENV_SEED_MAP = {..., "my_backend": {"credentials": {"api_key": "my_backend_api_key"}, "model": "my_backend_model_id"}}
```

The referenced `my_backend_api_key` / `my_backend_model_id` fields must exist on `Settings`.

Update `_model_id_for_backend()` in `ai/service.py` — its dict literal maps backend name to default model setting for AI-result traceability; add `"my_backend": settings.my_backend_model_id` alongside the other four.

Live connection test and model listing (recommended). Mirror the existing pattern in two files: `backend/app/ai/connection_test.py` (add a `_check_my_backend(credentials, model)` handler, register it in `test_ai_connection()` 's `handlers` dict, reached from `POST /api/v1/ai/test`, testing candidate credentials only) and `backend/app/api/routes/ai_config.py` (add `"my_backend"` to `_STATIC_MODEL_LISTS`, or implement live discovery in `POST /api/v1/ai/{backend}/models` following Groq's live `GET /models` or Ollama's live `GET {base_url}/api/tags`).

Frontend mirror. `frontend/lib/types.ts` and `lib/api.ts` do not introspect `AI_BACKENDS` — they are a manually-maintained mirror. Add the new name wherever the existing five are enumerated (e.g. `frontend/app/providers/page.tsx`, `AiQuickSwitch`), or it will be fully functional server-side but unreachable from the UI.

---

3. Adding a New API Endpoint

Every route lives under `backend/app/api/routes/`, one file per resource area, registered exactly once in `backend/app/main.py`. The simplest existing router to use as a template is `backend/app/api/routes/pivot.py` — one `GET` route, one permission check, one 404 case.

Create or extend a router file. For a new resource area, create `backend/app/api/routes/my_resource.py`:

```
from fastapi import APIRouter, Depends, HTTPException
from app.auth.rbac import CurrentUser, require_permission
from app.core.db import get_db

router = APIRouter(prefix="/my-resource", tags=["my-resource"])

@router.get("/{item_id}")
async def get_item(item_id: str, db=Depends(get_db),
                   user: CurrentUser = Depends(require_permission("my_resource:read"))):
    ...
```

For a new operation on an *existing* resource, add the function to the existing file (`cases.py` , `lookup.py` , etc.) instead — the pattern every multi-endpoint file already follows (all ten `cases.py` endpoints share one `router` and its `_load_case()` helper).

Gate it with `require_permission(...)`. This dependency factory (`backend/app/auth/rbac.py`) first resolves `get_current_user` (Bearer JWT, `401` if missing/invalid/inactive), then checks `permission in ROLE_PERMISSIONS.get(user.role, set())` , raising `403` otherwise. If the permission string doesn't already exist, add it to `ROLE_PERMISSIONS` in `backend/app/models/user.py` :

```
ROLE_PERMISSIONS: dict[Role, set[str]] = {
    Role.ADMIN: {..., "my_resource:read", "my_resource:write"},
    Role.ANALYST: {..., "my_resource:read"},
    Role.VIEWER: {..., "my_resource:read"},
}
```

This dict is the single source of truth for the entire authorization matrix. Only `/auth/register` , `/auth/login` , `/auth/refresh` (fully public) and `/auth/me` (bare `get_current_user` , no specific permission) skip this dependency.

Define request/response Pydantic schemas in `backend/app/schemas/` (e.g. `basket.py` , `case.py` , `lookup.py`) — separate from the AI-output schemas in `app/ai/schemas.py` / `app/ai/analysis_schemas.py` , which describe what an AI call must emit, not what the HTTP API accepts. Follow the existing plain- `BaseModel` style. Some endpoints deliberately accept a raw untyped `dict` instead (Copilot's `{"question": str, "notes": list[str]}` in `analysis.py` , basket-compare's `{"lookup_ids": list[str]}` in `basket.py`) — an accepted pattern for small, rarely-reused bodies, but a typed `BaseModel` is preferable for anything larger, since Pydantic then returns an automatic `422` on a bad shape.

Register the router in `main.py` , in the same style as the existing ten:

```
from app.api.routes import ai_config, analysis, auth, basket, cases, hunting, lookup, my_resource, pivot, providers, runtime
app.include_router(my_resource.router, prefix=settings.api_v1_prefix)
```

`settings.api_v1_prefix` (`"/api/v1"`) is applied uniformly here — the router's own `prefix=` only needs the resource-relative path. There is no other registration step: FastAPI derives the OpenAPI schema and Swagger UI (`/docs`) from whatever is registered here.

Update the frontend contract. Add the call to `frontend/lib/api.ts` (the frontend's sole backend client) and the response shape to `frontend/lib/types.ts` (a hand-maintained mirror of the backend Pydantic schemas, explicitly commented as needing to stay in sync). Nothing enforces this automatically.

---

4. Adding a Database Migration

Schema changes are managed with **Alembic** (`backend/alembic/`) against SQLAlchemy 2.0 async models. The migration history is a single linear chain of four revisions today: `660d2aa3bc20` (initial schema) → `a6d3ad2bb63c` (evidence, basket, case management) → `0f2dc283823e` (provider runtime config, audit log) → `2652d888a33f` (final assessment records, current head).

Change the model. Edit the relevant file under `backend/app/models/` . If it's a new table in a new module, make sure the module is imported from `backend/app/models/__init__.py` — its sole purpose is importing every model module so `Base.metadata` is fully populated before Alembic looks at it; a model class that exists but is never imported is invisible to autogenerate.

Generate the migration from `backend/` , with the database reachable (`Settings.database_url` , read by `alembic/env.py`):

```
cd backend
alembic revision --autogenerate -m "describe your change"
```

This produces a new file in `backend/alembic/versions/` , whose `down_revision` Alembic sets automatically to the current head (`2652d888a33f` as of this writing). **Always open and read the generated file before applying it** — autogenerate diffs model metadata against the database's current shape column-by-column, but does not reliably detect every kind of change (a column rename shows up as a drop-and-add unless hand-edited to `op.alter_column`) and never writes data-migration logic for you.

Apply it:

```
alembic upgrade head
```

This is exactly the command every container in this platform already runs on every startup, before the application server: both `docker-compose.yml` and `docker-compose.prod.yml` set the backend service's `command` to `sh -c "alembic upgrade head && uvicorn app.main:app ..."` . The running database's `alembic_version` table always records exactly which migration it was last brought up to date with — which is also the root of the Windows-packaging gotcha below.

The Windows installer "Can't locate revision" gotcha. This project's Windows installer does not run the application out of the git working tree. `windows/installer.iss` copies the entire `backend/` directory — including `backend/alembic/versions/*.py` — into the installed Program Files tree at install time (`Source: "{#RepoRoot}backend\*"; DestDir: "{app}\app\backend"; ...`). `windows/scripts/Common.ps1` documents the resulting split: read-only application code under Program Files (`$script:InstallDir`), writable runtime data under `C:\ProgramData\IOC Intelligence Platform\` (`$script:DataDir`). In production, `docker-compose.prod.yml` additionally strips every bind-mount from the backend service (`volumes: !reset []`) and rebuilds the backend image from scratch (`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`) — so the installed container's copy of `alembic/versions/` is baked into its image from whatever `.py` files existed in the **Program Files** copy at build time, not from the developer's working tree.

The failure mode: a new migration generated and applied (steps above) against a database from one copy of the code advances that database's `alembic_version` row to the new revision hash. If the *installed* Program Files copy of `backend/alembic/versions/` isn't also updated with that same `.py` file — by copying it in directly, or by rebuilding and reinstalling the installer — before that installed stack's backend container is next rebuilt or restarted, Alembic has no script matching the revision hash already recorded in the database. Because both compose files run `alembic upgrade head` as the very first step of the container's startup command, before `uvicorn` ever starts, this is not a soft failure: the entrypoint fails immediately with Alembic's `Can't locate revision identified by '<hash>'` error, and the backend never comes up, taking `uvicorn` down with it since the two commands are chained with `&&` .

The practical rule: every new file added to `backend/alembic/versions/` must reach every deployed copy of the application before, or at the same time as, any database that copy talks to is upgraded to that revision — by rebuilding and reinstalling the Windows installer package (which re-copies the current `backend/` tree per `installer.iss`), or, for a manual hotfix, by placing the new migration file directly into `{app}\app\backend\alembic\versions\` under Program Files and rebuilding the backend image before restarting the container.

Order	Revision	Down-revision	Created
1	`660d2aa3bc20`	(root)	`users`, `ioc_lookups`, `ai_summaries`, `correlation_edges`, `provider_results`
2	`a6d3ad2bb63c`	`660d2aa3bc20`	`cases`, `basket_items`, `case_iocs`, `case_notes`, `case_reports`, `evidence_items`
3	`0f2dc283823e`	`a6d3ad2bb63c`	`config_audit_log`, `provider_runtime_configs`
4 (head)	`2652d888a33f`	`0f2dc283823e`	`final_assessment_records`

A new migration's `down_revision` should always be `2652d888a33f` unless a teammate has already generated and merged a different migration first, in which case Alembic requires a merge revision (`alembic merge heads`) — not otherwise seen in this project's history, which has maintained a single linear chain to date.

Developer Troubleshooting Guide

This chapter is for someone working directly against the source tree at `ioc-intel-platform` -- running `docker compose` themselves, editing backend/frontend code, and running `pytest` -- not for an end user of the compiled Windows installer. It is the code-level counterpart to two other chapters covering adjacent but different ground: the user manual's System Health / Troubleshooting section documents symptoms an analyst can see in the browser with no code access, and the Windows Deployment chapter documents what the installer does mechanically. Below are the four confusions that most reliably cost a developer real time in this specific repository: two are Docker/Alembic build-vs-runtime mismatches, one is a Python import-naming collision in the test suite, and one is specific to how this project's Windows installer copies source rather than referencing it live. Every claim is cited to the file that actually causes the behavior.

1. "I edited the code, but the running container still behaves like the old version"

Symptom: A change to a route handler, provider connector, or frontend component has no effect after `docker compose restart backend` -- the container comes back up, logs look normal, but the old behavior persists.

Root cause: Both Dockerfiles build an image that copies the whole source tree in as a filesystem layer at build time, and that baked-in copy is what runs unless something overrides it at runtime. `backend/Dockerfile:9-12` does `COPY requirements.txt .` / `RUN pip install ...` / `COPY . .`; `frontend/Dockerfile:5-8` does the same pattern. Neither has a corresponding `.dockerignore` anywhere in the repo (root, `backend/`, or `frontend/` -- none exist), so it is an unfiltered copy of whatever is on disk at build time. `docker compose restart` and `docker compose up -d` (without `--build`) never re-run `COPY` -- they reuse the existing image layer cache. If that image predates your edit, your edit is not in the container, however many times it is restarted.

What masks this day to day is that the *development* compose file bind-mounts live source over the image's baked-in copy: `docker-compose.yml:104` and the equivalent blocks for `celery_worker` / `celery_beat` / `frontend` all declare `volumes: - ./<service>:/app`, and the backend command runs `uvicorn ... --reload` (`docker-compose.yml:106-108`), so the container's `/app` is the real working tree, live-reloaded. `docker-compose.prod.yml` -- the overlay the Windows installer runs via `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` (its own usage comment, line 13) -- explicitly does `volumes: !reset []` for `backend`, `celery_worker`, and `frontend` (lines 15, 19, 23), and says why in its header: "*This removes every source bind-mount so each service runs entirely from what was baked into its image at build time*" (lines 9-11).

Fix: Under the dev bind-mounted compose file, a restart is enough for backend/uvicorn changes (it reloads on file change) but Celery still needs `docker compose restart celery_worker` to re-import. Under `docker-compose.prod.yml`, or anything running through an installed Program Files copy (see §3), no restart ever picks up a source change -- run `docker compose build <service>` (or `up -d --build`) first.

2. `pytest` fails to collect, or errors on an argument mismatch, after importing a helper module

Symptom: Importing a "connection test" helper from application code into a test file causes pytest to try to run that imported function itself as a test -- it appears in the collected test list under its own name and fails with an argument-count error, because it takes production arguments (`provider_id`, `credentials`), not fixtures.

Root cause: Two real production functions in this codebase are named starting with `test_` because that is the correct English name for what they do -- they are not pytest tests: `backend/app/providers/connection_test.py:188` (`async def test_provider_connection(provider_id, credentials)`, behind `POST /api/v1/providers/{provider_id}/test`) and `backend/app/ai/connection_test.py:219` (`async def test_ai_connection(backend, credentials, model=None)`, behind `POST /api/v1/ai/test`). pytest's default discovery (`python_functions = test_*`) matches any name in a collected module's namespace starting with `test_`, including names brought in by a plain `import`, not just names defined locally -- and no `pytest.ini` / `pyproject.toml` / `setup.cfg` override of that default exists anywhere in `backend/`. A test file that did `from app.providers.connection_test import test_provider_connection` would have pytest try to collect and call it with no arguments, producing a confusing `TypeError` that reads like a test bug rather than a naming collision.

The codebase's own regression tests already hit this and fixed it the only correct way -- aliasing the import so the name in the test module's namespace no longer starts with `test_`: `backend/app/tests/unit/test_connection_test.py:13` (`from app.providers.connection_test import test_provider_connection as check_provider_connection`) and `backend/app/tests/unit/test_ai_connection_test.py:12` (same pattern for `check_ai_connection`). Every call site in both files then uses the aliased name.

Fix: Always alias an import of either function (`import ... as check_...`), mirroring the existing pattern. For any new helper you write that is meant to be imported near test code, avoid a `test_`-prefixed name in the first place. When in doubt, `pytest --collect-only <file>` after adding such an import to confirm nothing unexpected was picked up.

3. Changes made in the git working tree don't show up in the Windows-installed product

Symptom: A fix is confirmed working via `docker compose` in the actual repository checkout, but the installed HORIZON GRID copy (Start Menu shortcuts, `C:\Program Files\IOC Intelligence Platform\` — the on-disk folder name is unchanged from before the rebrand, deliberately) still exhibits the old behavior -- or, the reverse: a hand-edit made directly under `C:\Program Files\IOC Intelligence Platform\app\` disappears the next time the installer is re-run or the product is reconfigured/upgraded.

Root cause: The installer's `[Files]` section (`windows/installer.iss:77-78`) performs a **one-time file copy**, not a live link, from the repository into the install location: `Source: "{#RepoRoot}backend\*"; DestDir: "{app}\app\backend"; ...` and the equivalent for `frontend\*`. `windows/scripts/Common.ps1:8-12` documents the resulting layout in its own header comment: *"`C:\Program Files\IOC Intelligence Platform\app\` -- the actual repo (backend/, frontend/, docker-compose.yml, etc.) copied in by the Inno Setup installer. Read-mostly; docker compose builds images from here."* That last clause is the whole gotcha: every Docker image the installed product runs is built from the Program-Files copy, never from the developer's git checkout -- they are two independent copies of the same source tree, connected only by whatever the installer's copy step did at install time, with no filesystem link or sync between them. The installer's `Excludes:` clause (same lines, filtering `__pycache__` / `node_modules` /etc.) only strips generated artifacts from that copy; every real source file is copied in full, so a stale Program-Files copy is a complete, working, *old* snapshot rather than an obviously broken one -- which is exactly what makes this confusing to debug.

Fix: Develop and test against the git checkout's own `docker compose` for day-to-day work; only rebuild/rerun the installer when specifically validating the installed-product path. Never treat `{app}\app` as an editable copy -- any hand-edit there is lost on the next install/reconfigure and was never applied to the real source in git. If two people disagree about whether a bug is fixed, confirm which of the two independent copies each is actually testing.

4. Backend restart-loops, or a route fails against a column/table that doesn't exist, right after a rebuild

Symptom: After `docker compose build` / `up -d --build`, the `backend` container exits or loops during the `alembic upgrade head` step of startup, or starts fine but a specific endpoint throws `UndefinedColumnError` / `ProgrammingError` against a column the current model code expects.

Root cause: Both the dev and prod backend start commands run `alembic upgrade head` before `uvicorn` starts (`docker-compose.yml:106-108` ; identical minus `--reload` in `docker-compose.prod.yml:16-18`). `alembic upgrade head` only applies migration files physically present on disk inside that container at that moment -- it has no awareness of the model code beyond what those files describe. Because `backend/Dockerfile:12` 's `COPY . .` is what puts `backend/alembic/versions/*.py` into the image, and nothing filters that directory out, the same rebuild-vs-restart gap from §1 applies directly to migrations: a new revision file added to the repo but not yet baked into a rebuilt image is simply absent from that container's copy, not partially applied. `alembic upgrade head` exits cleanly having applied everything it *can see*, and the app starts against a schema one or more revisions behind what `backend/app/models/*.py` (imported into `Base.metadata` via `backend/alembic/env.py:10`) actually expects -- the failure then surfaces later, elsewhere, as a data-layer error rather than an obviously stale image.

This repository's migration history is a single linear chain with no branches and no verified drift between models and applied DDL (`660d2aa3bc20` initial schema -> `a6d3ad2bb63c` evidence/basket/case -> `0f2dc283823e` provider runtime config/audit -> `2652d888a33f` final assessment records, current head), which makes the failure easy to check for once suspected: the container's `alembic/versions/` should contain exactly those four files, and `alembic current` (run inside the container) should report `2652d888a33f (head)` .

Fix: Rebuild the backend and Celery images (they share the same build context, `docker-compose.yml:73,112,131`) after adding any file under `backend/alembic/versions/` -- `docker compose build backend celery_worker celery_beat` . Verify with `docker compose exec backend alembic current` against the known head. Never hand-edit an already-applied migration file -- Alembic tracks applied revisions by ID in the `alembic_version` table inside the `postgres_data` volume, not by file content, so editing a file after `upgrade head` already ran against it does nothing; write a new revision instead.

Quick Reference

Symptom	Root cause	Fix
Code edit has no effect after a restart	`docker compose restart` / `up -d` reuse the existing image; only a build re-runs `COPY . .` , and `docker-compose.prod.yml` removes the dev bind mounts that normally mask this	`docker compose build <service>` (or `up -d --build`) whenever the prod overlay or an installed copy is involved
`pytest` collects/fails on an imported `test_*` -named function	`test_provider_connection` (`app/providers/connection_test.py:188`) and `test_ai_connection` (`app/ai/connection_test.py:219`) are production functions, not tests, but match pytest's default `python_functions = test_*` discovery once imported into a test module	Alias the import (`... as check_provider_connection` / `check_ai_connection`), as already done in `test_connection_test.py:13` and `test_ai_connection_test.py:12`
Installed Windows product doesn't reflect a git-checkout change (or vice versa)	Installer copies source into `C:\Program Files\IOC Intelligence Platform\app\` once, at install time (`installer.iss:77-78`); Docker builds for the installed product always come from that copy, never from the git checkout (`Common.ps1:8-12`)	Develop/test against the git checkout's own `docker compose` ; only rebuild/rerun the installer to validate the installed-product path; never hand-edit files under Program Files
Backend restart-loops, or a route errors on a missing column/table right after a rebuild	`alembic upgrade head` (`docker-compose.yml:106-108`) only sees migration files baked into the image by `Dockerfile:12` 's `COPY . .` ; a new revision added to the repo but not yet rebuilt into the image is silently absent, not partially applied	Rebuild backend/Celery images after any addition under `backend/alembic/versions/` ; verify with `docker compose exec backend alembic current` against the known head (`2652d888a33f`)

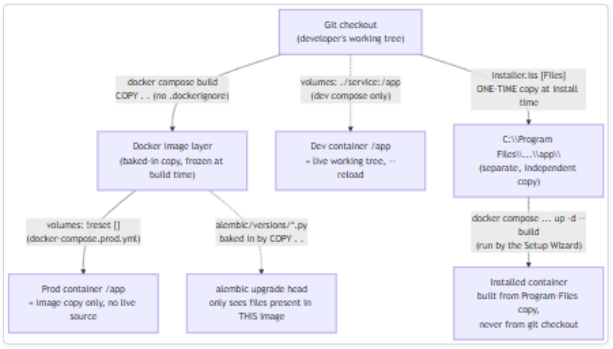

Figure 6 — Diagram: Quick Reference

Diagram: Build-time vs. runtime source of truth -- git checkout, Docker image layer, bind mount, and the separate Program-Files install copy. Four independent snapshots of the same source tree can exist at once (git checkout, dev image, prod image, Program Files copy), and only the dev-compose bind mount keeps any of them live-synced to an editor.

Summary

All four issues trace back to one fact, stated once so it needn't repeat: this project's Docker images are built by copying a directory tree (`COPY . .` , with no `.dockerignore` anywhere to filter it) at a specific point in time, and nothing about `restart` or a plain `up -d` re-triggers that copy. The development compose file's bind mounts make this mostly invisible day to day, which is exactly why it resurfaces, confusingly, the moment work moves to the production overlay, to Alembic migrations specifically, to the separately-copied Windows-installed product, or to a Python import that happens to start with the five characters `test_` .